

MIAO API

1 Overview

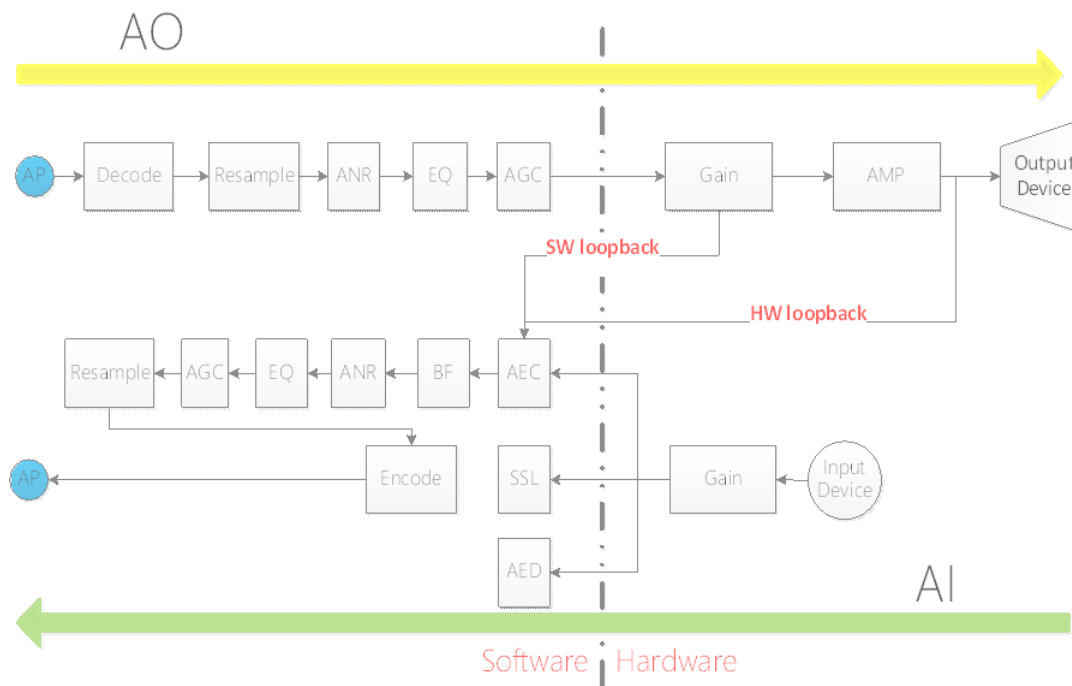
1.1. Module Description

Audio Output (AO): It mainly implements functions such as configuring and enabling audio output devices, sending audio frame data, and processing acoustic algorithms.

Acoustic algorithm processing mainly includes: resampling, noise reduction, high-pass filtering, equalizer, automatic gain control, etc.

In API versions 2.17 and later, MI_AO no longer contains related algorithm functions. Please check the version information in `mi_ao.h` of the SDK.

1.2. Flow Diagram



Picture 1-1

On API versions 2.17 and later, MI_AI/MI_AO no longer contains related algorithm functions, and only maintains basic capture/playback functions.

1.3. Keyword Description

- Device

Different from the Device concept of other modules, AO's Device refers to the path from the chip audio codec to external devices, such as Line out/I2S TX/HDMI, etc.

- Channel

The Channel of AO refers to the number of channels on the software, and a channel corresponds to several channels on the hardware, which is determined by MI_AUDIO_SoundMode_e.

- SRC

SRC (Sample Rate Conversion), namely resample, resampling.

- AGC

AGC (Automatic Gain Control), automatic gain control, is used to control the output gain.

- EQ

EQ (Equalizer), equalizer processing, is used to process specific frequency bands.

- ANR

ANR(Acoustic Noise Reduction) , 降噪 , 用于去除环境中持续存在 , 频率固定的噪声 。

- HPF

HPF(High-Pass Filtering) , 高通滤波

1.4. Device Id

表1-1 AO Dev ID和各个芯片物理设备的对照表

	0	1	2	3	4
Pretzel	Line out(DAC0/1)	I2s Tx			
Macaron	Line out(DAC0/1)	I2s Tx			
Taiyaki	Line out(DAC0/1)	I2s Tx	Hdmi	Hdmi + Line out	
Takoyaki	Line out(DAC0/1)	I2s Tx	Hdmi	Hdmi + Line out	
Pudding	Line out(DAC0/1)	I2s Tx			
Ispahan	Line out(DAC0/1)	I2s Tx			
Ikayaki	Line out(DAC0/1)	I2s Tx	DAC0	DAC1	HeadPhone

表1-2 不同系列chip的规格差异

	Line out	I2S Tx	Hdmi	HeadPhone
Pretzel	支持2路 8/16/32/48kHz 采样率	支持标准I2S模式和 TDM模式，TDM模式 可扩展到8路，支援 4-wire 和6-wire 模 式，可提供Mclk。支 持8/16/32/48kHz采 样率。	不支持	不支持
Macaron	支持2路 8/16/32/48kHz 采样率	只支持标准I2S模 式，且只能作为 master，仅支持4- wire 模式，不可提供	不支持	不支持
	Line out		Hdmi	HeadPhone

			支持 Mclk。支持 8/16/32/48kHz采样 率。	
Taiyaki	支持2路 8/16/32/48kHz 采样率	只支持标准I2S模 式，且只能作为 master，仅支持4- wire 模式，不可提供 Mclk。支持 8/16/32/48kHz采样 率。	支持	不支持
Takoyaki	支持2路 8/16/32/48kHz 采样率	只支持标准I2S模 式，且只能作为 master，仅支持4- wire 模式，不可提供 Mclk。支持 8/16/32/48kHz采样 率。	支持	不支持
Pudding	支持2路 8/16/32/48kHz 采样率	支持标准I2S模式和 TDM模式，TDM模式 可扩展到8路，支援 4-wire 和6-wire 模 式，可提供Mclk。支 持8/16/32/48kHz采 样率。	不支持	不支持
Ispahan	支持2路 8/16/32/48kHz 采样率	只支持标准I2S模 式，且只能作为 master，仅支持4- wire 模式，不可提供 Mclk。支持 8/16/32/48kHz采样 率。	不支持	不支持
Ikayaki	支持3路 8/16/32/48kHz 采样率	支持标准I2S模式和 TDM模式，TDM模式 可扩展到8路，支援 4-wire 和6-wire 模 式，可提供Mclk。支 持8/16/32/48kHz采 样率。	不支持	支持

2. API 参考

2.1. 功能模块API

表2-1

API名	功能
MI_AO_SetPubAttr [#22-mi_ao_setpubattr]	设置AO设备属性
MI_AO_GetPubAttr [#23-mi_ao_getpubattr]	获取AO设备属性
MI_AO_Enable [#24-mi_ao_enable]	启用AO设备
MI_AO_Disable [#25-mi_ao_disable]	禁用AO设备
MI_AO_EnableChn [#26-mi_ao_enablechn]	启用AO通道
MI_AO_DisableChn [#27-mi_ao_disablechn]	禁用AO通道
MI_AO_SendFrame [#28-mi_ao_sendframe]	发送音频帧
MI_AO_EnableReSmp [#29-mi_ao_enableresmp]	启用AO 重采样
MI_AO_DisableReSmp [#210-mi_ao_disableresmp]	禁用AO 重采样。
MI_AO_PauseChn [#211-mi_ao_pausechn]	暂停AO 通道
MI_AO_ResumeChn [#212-mi_ao_resumechn]	恢复AO 通道
MI_AO_ClearChnBuf [#213-mi_ao_clearchnbuf]	清除AO通道中当前的音频数据缓存
MI_AO_QueryChnStat [#214-mi_ao_querychnstat]	查询AO 通道中当前的音频数据缓存状态
API名	功能

MI_AO_SetVolume [#215-mi_ao_setvolume]	设置AO 设备/通道音量大小
MI_AO_GetVolume [#216-mi_ao_getvolume]	获取AO 设备/通道音量大小
MI_AO_SetMute [#217-mi_ao_setmute]	设置AO设备/通道静音状态
MI_AO_GetMute [#218-mi_ao_getmute]	获取AO设备/通道静音状态
MI_AO_ClrPubAttr [#219-mi_ao_clrpubattr]	清除AO设备属性
MI_AO_SetVqeAttr [#220-mi_ao_setvqeattr]	设置AO的声音质量增强功能相关属性
MI_AO_GetVqeAttr [#221-mi_ao_getvqeattr]	获取AO的声音质量增强功能相关属性
MI_AO_EnableVqe [#222-mi_ao_enablevqe]	使能AO的声音质量增强功能
MI_AO_DisableVqe [#223-mi_ao_disablevqe]	禁用AO的声音质量增强功能
MI_AO_SetAdecAttr [#224-mi_ao_setadecattr]	设置AO解码功能相关属性
MI_AO_GetAdecAttr [#225-mi_ao_getadecattr]	获取AO解码功能相关属性。
MI_AO_EnableAdec [#226-mi_ao_enableadec]	使能AO解码功能。
MI_AO_DisableAdec [#227-mi_ao_disableadec]	禁用AO解码功能。
MI_AO_SetChnParam [#228-mi_ao_setchnparam]	设置AO通道属性
MI_AO_GetChnParam [#229-mi_ao_getchnparam]	获取AO通道属性
MI_AO_SetSrcGain [#230-mi_ao_setsrcgain]	设置AO设备的回声参考增益
MI_AO_InitDev [#231-mi_ao_initdev]	初始化AO设备
MI_AO_DeinitDev [#232-mi_ao_deinitdev]	反初始化AO设备
MI_AO_DupChn [#233-mi_ao_dupchn]	同步AO通道状态
MI_AO_SetLRVolume [#234-mi_ao_setlrvolume]	设置AO 设备左右声道音量大小
MI_AO_GetLRVolume [#235-mi_ao_getlrvolume]	获取AO 设备左右声道音量大小

MI_AO_DevisEnable [#236-mi_ao_devisenable]	获取AO设备的使能状态
MI_AO_ChnlIsEnable [#237-mi_ao_chnisenable]	获取AO通道的使能状态

2.2. MI_AO_SetPubAttr

- 功能
设置AO设备属性。
- 语法

```
MI_S32 MI_AO_SetPubAttr(MI_AUDIO_DEV AoDevId, MI_AUDIO_Attr_t *pstAttr);
```

- 形参

表2-2

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
pstAttr	AO设备属性指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so

- 注意

音频输出设备属性包括采样率、采样精度、工作模式、每帧的采样点数、通道数目和I2S的配置参数。若需要对接Codec，这些属性应与待对接Codec 要

求一致。

- 采样率

采样率指一秒内的采样点数，采样率越高表明失真度越小，处理的数据量也就随之增加。一般来说语音使用8k 采样率，音频使用32k或以上的采样率；

目前仅支持8/11.025/12/16/22.05/24/32/44.1/48kHz采样率。若需要对接Codec，设置时请确认对接的Audio Codec 是否支持所要设定的采样率。

- 采样精度

采样精度指某个通道的采样点数据宽度。目前采样精度仅支持16bit。

- 工作模式

音频输入目前支持I2S主模式、I2S从模式、Tdm主模式、Tdm从模式，但芯片支持的工作模式不尽相同。

- 每帧的采样点数

当音频采样率较高时，建议相应地增加每帧的采样点数目。若发现采集到的声音断续，则需要增大每帧的采样点数。

- 通道数目

通道数目指软件概念上的声道数，而一个声道具体对应几个硬件上的声道，由MI_AUDIO_SoundMode_e决定。

- I2S的配置参数

I2S的配置参数指定I2S Mclk的频率、I2S传输的数据格式、I2S使用的是4-wire mode还是6-wire mode，I2S的Tdm slot数以及I2S收发数据的位宽。

- 举例

下面的代码实现初始化AO设备、送一帧音频数据、反初始化AO设备

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Attr_t stGetAttr;  
4. MI_AUDIO_DEV AoDevId = 0;  
5. MI_AO_CHN AoChn = 0;
```

```
6.  MI_U8 u8Buf[1024];

7.  MI_AUDIO_Frame_t stAoSendFrame;

8.  stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;

9.  stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;

10. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;

11. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;

12. stAttr.u32PtNumPerFrm = 1024;

13. stAttr.u32ChnCnt = 1;

14. MI_SYS_Init();

15. /* set ao public attr */

16. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);

17. if(MI_SUCCESS != ret)

18. {

19.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);

20.     return ret;

21. }

22. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);

23. if(MI_SUCCESS != ret)

24. {

25.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);

26.     return ret;

27. }

28. /* enable ao device */

29. ret = MI_AO_Enable(AoDevId);
```

```
30. if(MI_SUCCESS != ret)

31. {

32.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);

33.     return ret;

34. }

35. ret = MI_AO_EnableChn(AoDevId, AoChn);

36. if (MI_SUCCESS != ret)

37. {

38.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId, AoChn,

39.         return ret;

40. }

41. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));

42. stAoSendFrame.u32Len = 1024;

43. stAoSendFrame.apVirAddr[0] = u8Buf;

44. stAoSendFrame.apVirAddr[1] = NULL;

45. do{

46.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame, -1);

47. }while(ret == MI_AO_ERR_NOBUF);

48. ret = MI_AO_DisableChn(AoDevId, AoChn);

49. if (MI_SUCCESS != ret)

50. {

51.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId, AoChn,
```

```

52.     return ret;

53. }

54. /* disable ao device */

55. ret = MI_AO_Disable(AoDevId);

56. if(MI_SUCCESS != ret)

57. {

58.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);

59.     return ret;

60. }

61. MI_SYS_Exit();

```

2.3. MI_AO_GetPubAttr

- 功能

获取AO设备属性。

- 语法

```

MI_S32 MI_AO_GetPubAttr(MI_AUDIO_DEV AoDevId, MI_AUDIO_Attr_t
*pstAttr);

```

- 形参

表2-3

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
pstAttr	AO设备属性指针。	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码 \[#jump\]](#)。
- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`
- 注意
 - 获取的属性为前一次配置的属性。
 - 如果从来没有配置过属性，则返回失败。
- 举例

请参考[MI_AO_SetPubAttr \[#22-mi_ao_setpubattr\]](#)举例部分。

2.4. MI_AO_Enable

- 功能
启用AO设备。
- 语法

```
MI_S32 MI_AO_Enable(MI_AUDIO_DEV AoDevId);
```

- 形参

表2-4

参数名称	描述	输入/输出
AoDevId	音频设备号	输入

- 返回值
 - 0 成功
 - 非0 失败，参照[错误码 \[#jump\]](#)。
- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`
- 注意
 - 必须在启用前配置AO设备属性，否则返回属性未配置错误
 - 如果AO设备已经处于启用状态，则直接返回成功。
- 举例

请参考[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr] 的举例部分。

2.5. MI_AO_Disable

- 功能
禁用AO设备。
- 语法

```
MI_S32 MI_AO_Disable(MI_AUDIO_DEV AoDevId);
```

- 形参

表2-5

参数名称	描述	输入/输出
AoDevId	音频设备号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`
- 注意

- 如果AO设备已经处于禁用状态，则直接返回成功
- 禁用AO设备前必须先禁用该设备下已启用的所有AO通道
- 只要是修改设备属性，都必须先调用MI_AO_Disable，再重新设置设备属性，使用AO设备/通道。
- 举例

请参考[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr] 的举例部分。

2.6. MI_AO_EnableChn

- 功能
启用AO通道
- 语法

```
MI_S32 MI_AO_EnableChn(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn);
```

- 形参

表2-6

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值
 - 0 成功
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意

- 启用AO通道前，必须先启用其所属的AO设备，否则返回设备未启动的错误码
 - 举例
- 请参考[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr] 的举例部分。

2.7. MI_AO_DisableChn

- 功能
- 禁用AI通道
- 语法

```
MI_S32 MI_AO_DisableChn( AoDevId, AoChn);
```

- 形参

表2-7

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
 - 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`
 - 注意
- 在禁用AO通道前，需要将该通道上使能的音频算法禁用。
- 举例

请参考[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr] 的举例部分。

2.8. MI_AO_SendFrame

- 功能

发送AO 音频帧。

- 语法

```
MI_S32 MI_AO_SendFrame(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
                        *pstData, MI_S32 s32MilliSec);
```

- 形参

表2-8

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号。仅支持使用通道0。	输入
pstData	音频帧结构体指针。	输入
s32MilliSec	设置数据的超时时间 -1 表示阻塞模式，无数据时一直等待； 0 表示非阻塞模式，无数据时则报错返回； >0表示阻塞s32MilliSec毫秒，超时则报错返回。	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

- 注意

- `s32MilliSec` 的值必须大于等于-1，等于-1 时采用阻塞模式发送数据，等于0时采用非阻塞模式发送数据，大于0时，阻塞`s32MilliSec` 毫秒后，仍然发送不成功则返回超时并报错
- 调用该接口发送音频帧到AO 输出时，必须先使能对应的AO 通道。
- 举例

请参考[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr] 的举例部分。

2.9. MI_AO_EnableReSmp

- 功能
启用AO 重采样
- 语法

```
MI_S32 MI_AO_EnableReSmp(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_AUDIO_SampleRate_e eInSampleRate);
```

- 形参

表2-9

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号	输入
eInSampleRate	音频重采样的输入采样率。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ao.h`

- 库文件: libmi_ao.a/libmi_ao.so libSRC_LINUX.so
- 注意
 - 应该在启用AO通道之后，调用此接口启用重采样功能。
 - 在API版本2.17及以后，已不支持此接口。
- 举例

下面的代码实现开启和关闭重采样，输入数据为16K，重采样至8K播放。

```
1. MI_S32 ret;
2. MI_AUDIO_Attr_t stAttr;
3. MI_AUDIO_Attr_t stGetAttr;
4. MI_AUDIO_DEV AoDevId = 0;
5. MI_AO_CHN AoChn = 0;
6. MI_U8 u8Buf[1024];
7. MI_AUDIO_Frame_t stAoSendFrame;
8. MI_AUDIO_SampleRate_e eInSampleRate =
E_MI_AUDIO_SAMPLE_RATE_16000;
9.
10. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
11. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
12. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
13. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
14. stAttr.u32PtNumPerFrm = 1024;
15. stAttr.u32ChnCnt = 1;
16.
17. MI_SYS_Init();
18.
19. /* set ao public attr*/
20. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
21. if(MI_SUCCESS != ret)
22. {
23.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
24.     return ret;
25. }
26.
27. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
28. if(MI_SUCCESS != ret)
29. {
30.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
31.     return ret;
32. }
33.
34. /* enable ao device*/
35. ret = MI_AO_Enable(AoDevId);
36. if(MI_SUCCESS != ret)
```

```
37. {
38.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
39.     return ret;
40. }
41.
42. ret = MI_AO_EnableChn(AoDevId, AoChn);
43. if (MI_SUCCESS != ret)
44. {
45.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
46.     return ret;
47. }
48.
49. ret = MI_AO_EnableReSmp(AoDevId, AoChn, eInSampleRate);
50. if (MI_SUCCESS != ret)
51. {
52.     printf("enable resample ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
53.     return ret;
54. }
55.
56. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
57. stAoSendFrame.u32Len = 1024;
58. stAoSendFrame.apVirAddr[0] = u8Buf;
59. stAoSendFrame.apVirAddr[1] = NULL;
60.
61. do{
62.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
63. }while(ret == MI_AO_ERR_NOBUF);
64.
65. ret = MI_AO_DisableReSmp(AoDevId, AoChn);
66. if (MI_SUCCESS != ret)
67. {
68.     printf("disable resample ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
69.     return ret;
70. }
71.
72. ret = MI_AO_DisableChn(AoDevId, AoChn);
73. if (MI_SUCCESS != ret)
74. {
75.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
76.     return ret;
77. }
78.
79. /* disable ao device */
80. ret = MI_AO_Disable(AoDevId);
```

```
81. if(MI_SUCCESS != ret)
82. {
83.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
84.     return ret;
85. }
86.
87. MI_SYS_Exit();
```

2.10. MI_AO_DisableReSmp

- 功能

禁用AO 重采样

- 语法

MI_S32 MI_AO_DisableReSmp(AoDevId, MI_AO_CHN [#m34-i_ao_chn]
AoChn);

- 形参

表2-10

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.a/libmi_ao.so libSRC_LINUX.so

- 注意

- 不再使用AO重采样功能的话，应该调用此接口将其禁用。

- 在API版本2.17及以后，已不支持此接口。
- 举例

请参考[MI_AO_EnableReSmp](#) [#29-mi_ao_enableresmp]举例部分。

2.11. MI_AO_PauseChn

- 功能

暂停AO 通道

- 语法

```
MI_S32 MI_AO_PauseChn(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn);
```

- 形参

表2-11

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

- 注意

- AO 通道为禁用状态时，不允许调用此接口暂停AO通道。

- 举例

下面的代码实现暂停和恢复AO播放。

```
1. MI_S32 ret;
2. MI_AUDIO_Attr_t stAttr;
3. MI_AUDIO_Attr_t stGetAttr;
4. MI_AUDIO_DEV AoDevId = 0;
5. MI_AO_CHN AoChn = 0;
6. MI_U8 u8Buf[1024];
7. MI_AUDIO_Frame_t stAoSendFrame;
8.
9. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
10. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
11. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
12. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
13. stAttr.u32PtNumPerFrm = 1024;
14. stAttr.u32ChnCnt = 1;
15.
16. MI_SYS_Init();
17.
18. /* set ao public attr*/
19. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
20. if(MI_SUCCESS != ret)
21. {
22.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
23.     return ret;
24. }
25.
26. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
27. if(MI_SUCCESS != ret)
28. {
29.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
30.     return ret;
31. }
32.
33. /* enable ao device*/
34. ret = MI_AO_Enable(AoDevId);
35. if(MI_SUCCESS != ret)
36. {
37.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
38.     return ret;
39. }
40.
41. ret = MI_AO_EnableChn(AoDevId, AoChn);
42. if (MI_SUCCESS != ret)
43. {
44.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
45.     return ret;
46. }
47.
```

```

48. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
49. stAoSendFrame.u32Len = 1024;
50. stAoSendFrame.apVirAddr[0] = u8Buf;
51. stAoSendFrame.apVirAddr[1] = NULL;
52.
53. do{
54.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
55. }while(ret == MI_AO_ERR_NOBUF);
56.
57. ret = MI_AO_PauseChn(AoDevId, AoChn);
58. if (MI_SUCCESS != ret)
59. {
60.     printf("pause ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
61.     return ret;
62. }
63.
64. ret = MI_AO_ResumeChn(AoDevId, AoChn);
65. if (MI_SUCCESS != ret)
66. {
67.     printf("resume ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
68.     return ret;
69. }
70.
71. ret = MI_AO_DisableChn(AoDevId, AoChn);
72. if (MI_SUCCESS != ret)
73. {
74.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
75.     return ret;
76. }
77.
78. /* disable ao device */
79. ret = MI_AO_Disable(AoDevId);
80. if(MI_SUCCESS != ret)
81. {
82.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
83.     return ret;
84. }
85.
86. MI_SYS_Exit();

```

2.12. MI_AO_ResumeChn

- 功能
恢复AO 通道。
- 语法

```
MI_S32 MI_AO_ResumeChn (MI_AUDIO_DEV AoDevId,    MI_AO_CHN
AoChn);
```

- 形参

表2-12

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意
 - AO 通道暂停后可以通过调用此接口重新恢复。
 - AO 通道为暂停状态或使能状态下，调用此接口返回成功；否则调用将返回错误。
- 举例

请参考[MI_AO_PauseChn](#) [#211-mi_ao_pausechn]举例部分。

2.13. MI_AO_ClearChnBuf

- 功能

清除AO 通道中当前的音频数据缓存。

- 语法

```
MI_S32 MI_AO_ClearChnBuf (MI_AUDIO_DEV AoDevId,    MI_AO_CHN
AoChn);
```

- 形参

表2-13

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号	输入

- 返回值

- 0 成功
- 非0 失败，参照[错误码](#) [#jump]

- 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.a/libmi_ao.so

- 注意

- AO通道成功启用后再调用此接口。

- 举例

下面的代码实现获取AO通道缓存情况并清除缓存。

```
1. MI_S32 ret;
2. MI_AUDIO_Attr_t stAttr;
3. MI_AUDIO_Attr_t stGetAttr;
4. MI_AUDIO_DEV AoDevId = 0;
5. MI_AO_CHN AoChn = 0;
6. MI_U8 u8Buf[1024];
7. MI_AUDIO_Frame_t stAoSendFrame;
8. MI_AO_ChnState_t stStatus;
```

```
9.
10. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
11. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
12. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
13. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
14. stAttr.u32PtNumPerFrm = 1024;
15. stAttr.u32ChnCnt = 1;
16.
17. MI_SYS_Init();
18.
19. /* set ao public attr*/
20. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
21. if(MI_SUCCESS != ret)
22. {
23.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
24.     return ret;
25. }
26.
27. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
28. if(MI_SUCCESS != ret)
29. {
30.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
31.     return ret;
32. }
33.
34. /* enable ao device*/
35. ret = MI_AO_Enable(AoDevId);
36. if(MI_SUCCESS != ret)
37. {
38.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
39.     return ret;
40. }
41.
42. /* enable ao chn */
43. ret = MI_AO_EnableChn(AoDevId, AoChn);
44. if (MI_SUCCESS != ret)
45. {
46.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
47.     return ret;
48. }
49.
50. /* send frame */
51. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
52. stAoSendFrame.u32Len = 1024;
53. stAoSendFrame.apVirAddr[0] = u8Buf;
54. stAoSendFrame.apVirAddr[1] = NULL;
55.
56. do{
```

```

57.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
58. }while(ret == MI_AO_ERR_NOBUF);
59.
60. /* get chn stat */
61. ret = MI_AO_QueryChnStat(AoDevId, AoChn, &stStatus);
62. if (MI_SUCCESS != ret)
63. {
64.     printf("query chn status ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
65.     return ret;
66. }
67.
68. /* clear chn buf */
69. ret = MI_AO_ClearChnBuf(AoDevId, AoChn);
70. if (MI_SUCCESS != ret)
71. {
72.     printf("clear chn buf ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
73.     return ret;
74. }
75.
76. /* disable ao chn */
77. ret = MI_AO_DisableChn(AoDevId, AoChn);
78. if (MI_SUCCESS != ret)
79. {
80.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
81.     return ret;
82. }
83.
84. /* disable ao device */
85. ret = MI_AO_Disable(AoDevId);
86. if(MI_SUCCESS != ret)
87. {
88.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
89.     return ret;
90. }
91.
92. MI_SYS_Exit();

```

2.14. MI_AO_QueryChnStat

- 功能

查询AO 通道中当前的音频数据缓存状态。

- 语法

```
MI_S32 MI_AO_QueryChnStat(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_AO_ChnState_t *pstStatus);
```

- 形参

表2-14

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入
pstStatus	缓存状态结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

- 注意

- AO通道成功启用后再调用此接口。

- 举例

请参考[MI_AO_ClearChnBuf](#) [213-mi_ao_clearchnbuf] 举例部分。

2.15. MI_AO_SetVolume

- 功能

设置AO设备/通道音量大小。

• 语法

```
MI_AO API Version <= 2.13:
MI_S32 MI_AO_SetVolume(MI_AUDIO_DEV AoDevId, MI_S32
s32VolumeDb);

MI_AO API Version为2.14:
MI_S32 MI_AO_SetVolume(MI_AUDIO_DEV AoDevId, MI_S32
s32VolumeDb, MI_AO_GainFading_e eFading);

MI_AO API Version >= 2.15:
MI_S32 MI_AO_SetVolume(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
MI_S32 s32VolumeDb, MI_AO_GainFading_e eFading);
```

• 形参

表2-15

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号	输入
s32VolumeDb	音频设备音量大小 (-60 – 30 , 以dB为单位)	输入
eFading	音频增益变化的速度	输入

• 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

• 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

• 注意

- AO设备成功启用后再调用此接口。

• 举例

下面的代码实现设置或获取AO的音量参数

```
1.  MI_S32 ret;
2.  MI_AUDIO_Attr_t stAttr;
3.  MI_AUDIO_Attr_t stGetAttr;
4.  MI_AUDIO_DEV AoDevId = 0;
5.  MI_AO_CHN AoChn = 0;
6.  MI_U8 u8Buf[1024];
7.  MI_AUDIO_Frame_t stAoSendFrame;
8.  MI_S32 s32VolumeDb = 0, s32VolumeDb1 = 0;
9.
10. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
11. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
12. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
13. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
14. stAttr.u32PtNumPerFrm = 1024;
15. stAttr.u32ChnCnt = 1;
16.
17. MI_SYS_Init();
18.
19. /* set ao public attr*/
20. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
21. if(MI_SUCCESS != ret)
22. {
23.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
24.     return ret;
25. }
26.
27. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
28. if(MI_SUCCESS != ret)
29. {
30.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
31.     return ret;
32. }
33.
34. /* enable ao device*/
35. ret = MI_AO_Enable(AoDevId);
36. if(MI_SUCCESS != ret)
37. {
38.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
39.     return ret;
40. }
41.
42. /* enable ao chn */
43. ret = MI_AO_EnableChn(AoDevId, AoChn);
44. if (MI_SUCCESS != ret)
45. {
46.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
```

```

AoChn, ret);
47.     return ret;
48. }
49.
50. /* set ao volume */
51. // ret = MI_S32 MI_AO_SetLRVolume(AoDevId, AoChn,
s32VolumeDb, s32VolumeDb1, E_MI_AO_GAIN_FADING_OFF);
52. ret = MI_S32 MI_AO_SetVolume(AoDevId, AoChn, s32VolumeDb,
E_MI_AO_GAIN_FADING_OFF);
53. if (MI_SUCCESS != ret)
54. {
55.     printf("set volume ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
56.     return ret;
57. }
58.
59. /* get ao volume */
60. // ret = MI_S32 MI_AO_GetLRVolume(AoDevId, AoChn,
&s32VolumeDb, &s32VolumeDb1);
61. ret = MI_S32 MI_AO_GetVolume(AoDevId, AoChn, &s32VolumeDb);
62. if (MI_SUCCESS != ret)
63. {
64.     printf("get volume ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
65.     return ret;
66. }
67.
68.
69.
70. /* send frame */
71. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
72. stAoSendFrame.u32Len = 1024;
73. stAoSendFrame.apVirAddr[0] = u8Buf;
74. stAoSendFrame.apVirAddr[1] = NULL;
75.
76. do{
77.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
78. }while(ret == MI_AO_ERR_NOBUF);
79.
80.
81. /* disable ao chn */
82. ret = MI_AO_DisableChn(AoDevId, AoChn);
83. if (MI_SUCCESS != ret)
84. {
85.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
86.     return ret;
87. }

```

```

88.
89. /* disable ao device */
90. ret = MI_AO_Disable(AoDevId);
91. if(MI_SUCCESS != ret)
92. {
93.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
94.     return ret;
95. }
96.
97. MI_SYS_Exit();

```

2.16. MI_AO_GetVolume

- 功能

获取AO设备/通道音量大小。

- 语法

MI_AO API Version <= 2.14:

```
MI_S32 MI_AO_GetVolume(MI_AUDIO_DEV AoDevId, MI_S32
*ps32VolumeDb);
```

MI_AO API Version >= 2.15:

```
MI_S32 MI_AO_GetVolume(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
MI_S32 *ps32VolumeDb);
```

- 形参

表2-16

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号	输入
ps32VolumeDb	音频设备音量大小指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意
 - AO设备成功启用后再调用此接口。
- 举例

请参考[MI_AO_SetVolume](#) [#215-mi_ao_setvolume]的举例部分。

2.17. MI_AO_SetMute

- 功能

设置AO 设备/通道的静音状态。
- 语法

```
MI_AO API Version <= 2.14:
```

```
MI_S32 MI_AO_SetMute(MI_AUDIO_DEV AoDevId, MI_BOOL bEnable);
```

```
MI_AO API Version >= 2.15:
```

```
MI_S32 MI_AO_SetMute(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,  
MI_BOOL bEnable);
```

- 形参

表2-17

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号	输入
bEnable	音频设备是否启用静音 TRUE：启用静音功能；FALSE：关闭静音功能。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意
 - AO设备成功启用后再调用此接口。
- 举例

下面的代码实现获取和设置静音状态。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Attr_t stGetAttr;  
4. MI_AUDIO_DEV AoDevId = 0;  
5. MI_AO_CHN AoChn = 0;  
6. MI_U8 u8Buf[1024];  
7. MI_AUDIO_Frame_t stAoSendFrame;  
8. MI_BOOL bMute = TRUE;  
9.  
10. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;  
11. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;  
12. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;  
13. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;  
14. stAttr.u32PtNumPerFrm = 1024;  
15. stAttr.u32ChnCnt = 1;  
16.  
17. MI_SYS_Init();  
18.
```

```
19. /* set ao public attr*/
20. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
21. if(MI_SUCCESS != ret)
22. {
23.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
24.     return ret;
25. }
26.
27. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
28. if(MI_SUCCESS != ret)
29. {
30.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
31.     return ret;
32. }
33.
34. /* enable ao device*/
35. ret = MI_AO_Enable(AoDevId);
36. if(MI_SUCCESS != ret)
37. {
38.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
39.     return ret;
40. }
41.
42. /* enable ao chn */
43. ret = MI_AO_EnableChn(AoDevId, AoChn);
44. if (MI_SUCCESS != ret)
45. {
46.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
47.     return ret;
48. }
49.
50. /* set ao mute */
51. ret = MI_AO_SetMute(AoDevId, AoChn, bMute);
52. if (MI_SUCCESS != ret)
53. {
54.     printf("mute ao dev %d Chn%d err:0x%x\n", AoDevId,
AoChn, ret);
55.     return ret;
56. }
57.
58. /* get ao mute status */
59. ret = MI_AO_GetMute(AoDevId, AoChn, &bMute);
60. if (MI_SUCCESS != ret)
61. {
62.     printf("get mute status ao dev %d Chn %d err:0x%x\n",
AoDevId, AoChn, ret);
63.     return ret;
64. }
```

```

65.
66. /* send frame */
67. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
68. stAoSendFrame.u32Len = 1024;
69. stAoSendFrame.apVirAddr[0] = u8Buf;
70. stAoSendFrame.apVirAddr[1] = NULL;
71.
72. do{
73.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
74. }while(ret == MI_AO_ERR_NOBUF);
75.
76.
77. /* disable ao chn */
78. ret = MI_AO_DisableChn(AoDevId, AoChn);
79. if (MI_SUCCESS != ret)
80. {
81.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
82.     return ret;
83. }
84.
85. /* disable ao device */
86. ret = MI_AO_Disable(AoDevId);
87. if(MI_SUCCESS != ret)
88. {
89.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
90.     return ret;
91. }
92.
93. MI_SYS_Exit();

```

2.18. MI_AO_GetMute

- 功能

获取AO 设备/通道的静音状态。

- 语法

MI_AO API **Version** <= **2.14**:

```

MI_S32 MI_AO_GetMute(MI_AUDIO_DEV AoDevId, MI_BOOL
*pbEnable);

```

MI_AO API [Version](#) >= [2.15](#):

```
MI_S32 MI_AO_GetMute(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
MI_BOOL *pbEnable);
```

- 形参

表2-18

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号	输入
p**bEnable**	音频设备静音状态指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [[#jump](#)]。
- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`
- 注意
 - AO设备成功启用后再调用此接口。
- 举例

请参考[MI_AO_SetMute](#) [[#217-mi_ao_setmute](#)]举例部分

2.19. MI_AO_ClrPubAttr

- 功能

清除AO设备属性。
- 语法

```
MI_S32 MI_AO_ClrPubAttr(MI_AUDIO_DEV AoDevId);
```

- 形参

表2-19

参数名称	描述	输入/输出
AoDevId	音频设备号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意
 - 清除设备属性前，需要先停止设备。
- 举例

下面的代码实现设置和清除AO设备的属性。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_DEV AoDevId = 0;  
4.  
5. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;  
6. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;  
7. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;  
8. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;  
9. stAttr.u32PtNumPerFrm = 1024;  
10. stAttr.u32ChnCnt = 1;  
11.  
12. MI_SYS_Init();  
13.  
14. /* set ao public attr*/  
15. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);  
16. if(MI_SUCCESS != ret)  
17. {
```

```
18.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
19.     return ret;
20. }
21.
22. /* clear ao public attr */
23. ret = MI_AO_ClrPubAttr(AoDevId);
24. if (MI_SUCCESS != ret)
25. {
26.     printf("clear ao %d attr err:0x%x\n", AoDevId, ret);
27.     return ret;
28. }
29.
30. MI_SYS_Exit();
```

2.20. MI_AO_SetVqeAttr

- 功能

设置AO的声音质量增强功能相关属性。

- 语法

```
MI_S32 MI_AO_SetVqeAttr(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
MI_AO_VqeConfig_t *pstVqeConfig);
```

- 形参

表2-20

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入
pstVqeConfig	音频输出声音质量增强配置结构体指针	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so libAPC_LINUX.so
- 注意
 - 启用声音质量增强功能前必须先设置相对应AO通道的声音质量增强功能相关属性。
 - 设置AO的声音质量增强功能相关属性前，必须先使能对应的AO通道。
 - 相同AO 信道的声音质量增强功能不支持动态设置属性，重新设置AO通道的声音质量增强功能相关属性时，需要先关闭AO通道的声音质量功能，再设置AO通道的声音质量增强功能相关属性。
 - 在设置声音质量增强功能属性时，可通过配置相应的声音质量增强功能属性来选择使能其中的部分功能。
 - Vqe 所有算法均支持8K/16K，仅ANR/AGC/EQ支持48K。
 - 在API版本2.17及以后，已不支持此接口。
- 举例

下面的代码实现设置和使能声音质量增强功能。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Attr_t stGetAttr;  
4. MI_AUDIO_DEV AoDevId = 0;  
5. MI_AO_CHN AoChn = 0;  
6. MI_U8 u8Buf[1024];  
7. MI_AUDIO_Frame_t stAoSendFrame;  
8. MI_AO_VqeConfig_t stAoSetVqeConfig, stAoGetVqeConfig;  
9.  
10. MI_AUDIO_HpfConfig_t stHpfCfg = {  
11.     .eMode = E_MI_AUDIO_ALGORITHM_MODE_USER,  
12.     .eHpfFreq = E_MI_AUDIO_HPF_FREQ_150,  
13. };  
14.  
15. MI_AUDIO_AnrcConfig_t stAnrcCfg = {  
16.     .eMode = E_MI_AUDIO_ALGORITHM_MODE_MUSIC,  
17.     .u32NrIntensity = 15,  
18.     .u32NrSmoothLevel = 10,  
19.     .eNrSpeed = E_MI_AUDIO_NR_SPEED_MID,  
20. };  
21.
```

```

22. MI_AUDIO_AgcConfig_t stAgcCfg = {
23.     .eMode = E_MI_AUDIO_ALGORITHM_MODE_USER,
24.     .s32NoiseGateDb = -60,
25.     .s32TargetLevelDb = -3,
26.     .stAgcGainInfo = {
27.         .s32GainInit = 0,
28.         .s32GainMax = 20,
29.         .s32GainMin = 0,
30.     },
31.     .u32AttackTime = 1,
32.     .s16Compression_ratio_input = {-80, -60, -40, -25, 0},
33.     .s16Compression_ratio_output = {-80, -30, -15, -10,
-3},
34.     .u32DropGainMax = 12,
35.     .u32NoiseGateAttenuationDb = 0,
36.     .u32ReleaseTime = 3,
37. };
38.
39. MI_AUDIO_EqConfig_t stEqCfg = {
40.     .eMode = E_MI_AUDIO_ALGORITHM_MODE_DEFAULT,
41.     .s16EqGainDb = {[0 ... 128] = 3},
42. };
43.
44. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
45. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
46. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
47. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
48. stAttr.u32PtNumPerFrm = 1024;
49. stAttr.u32ChnCnt = 1;
50.
51. MI_SYS_Init();
52.
53. /* set ao public attr*/
54. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
55. if(MI_SUCCESS != ret)
56. {
57.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
58.     return ret;
59. }
60.
61. /* get ao public attr */
62. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
63. if(MI_SUCCESS != ret)
64. {
65.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
66.     return ret;
67. }
68.
69. /* enable ao device*/

```

```
70. ret = MI_AO_Enable(AoDevId);
71. if(MI_SUCCESS != ret)
72. {
73.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
74.     return ret;
75. }
76.
77. /* enable ao chn */
78. ret = MI_AO_EnableChn(AoDevId, AoChn);
79. if (MI_SUCCESS != ret)
80. {
81.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
82.     return ret;
83. }
84.
85. stAoSetVqeConfig.bAgcOpen = TRUE;
86. stAoSetVqeConfig.bAnrOpen = TRUE;
87. stAoSetVqeConfig.bEqOpen = TRUE;
88. stAoSetVqeConfig.bHpfOpen = TRUE;
89. stAoSetVqeConfig.s32FrameSample = 128;
90. stAoSetVqeConfig.s32WorkSampleRate =
E_MI_AUDIO_SAMPLE_RATE_8000;
91. memcpy(&stAoSetVqeConfig.stAgcCfg, &stAgcCfg,
sizeof(MI_AUDIO_AgcConfig_t));
92. memcpy(&stAoSetVqeConfig.stAnrCfg, &stAnrCfg,
sizeof(MI_AUDIO_Anrcfg_t));
93. memcpy(&stAoSetVqeConfig.stEqCfg, &stEqCfg,
sizeof(MI_AUDIO_EqConfig_t));
94. memcpy(&stAoSetVqeConfig.stHpfCfg, &stHpfCfg,
sizeof(MI_AUDIO_HpfConfig_t));
95.
96. /* set vqe attr */
97. ret = MI_AO_SetVqeAttr(AoDevId, AoChn, &stAoSetVqeConfig);
98. if (MI_SUCCESS != s32Ret)
99. {
100.     printf("set vqe attr ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
101.     return ret;
102. }
103.
104. /* get vqe attr */
105. ret = MI_AO_GetVqeAttr(AoDevId, AoChn,
&stAoGetVqeConfig);
106. if (MI_SUCCESS != s32Ret)
107. {
108.     printf("get vqe attr ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
109.     return ret;
```

```
110.     }
111.
112.     /* enable vqe attr */
113.     ret = MI_AO_EnableVqe(AoDevId, AoChn);
114.     if (MI_SUCCESS != s32Ret)
115.     {
116.         printf("enable vqe ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
117.         return ret;
118.     }
119.
120.     /* send frame */
121.     memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
122.     stAoSendFrame.u32Len = 1024;
123.     stAoSendFrame.apVirAddr[0] = u8Buf;
124.     stAoSendFrame.apVirAddr[1] = NULL;
125.
126.     do{
127.         ret = MI_AO_SendFrame(AoDevId, AoChn,
&stAoSendFrame, -1);
128.     }while(ret == MI_AO_ERR_NOBUF);
129.
130.     /* disable vqe attr */
131.     ret = MI_AO_DisableVqe(AoDevId, AoChn);
132.     if (MI_SUCCESS != s32Ret)
133.     {
134.         printf("disable vqe ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
135.         return ret;
136.     }
137.
138.     /* disable ao chn */
139.     ret = MI_AO_DisableChn(AoDevId, AoChn);
140.     if (MI_SUCCESS != ret)
141.     {
142.         printf("disable ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
143.         return ret;
144.     }
145.
146.     /* disable ao device */
147.     ret = MI_AO_Disable(AoDevId);
148.     if(MI_SUCCESS != ret)
149.     {
150.         printf("disable ao dev %d err:0x%x\n", AoDevId,
ret);
151.         return ret;
152.     }
```

```
153.  
154.     MI_SYS_Exit();
```

2.21. MI_AO_GetVqeAttr

- 功能
获取AO的声音质量增强功能相关属性。
- 语法

```
MI_S32 MI_AO_GetVqeAttr(MI_AUDIO_DEV AoDevId,    MI_AO_CHN  
AoChn, MI_AO_VqeConfig_t  *pstVqeConfig);
```

- 形参

表2-21

参数名称	描述	输入/输出
AiDevId	音频设备号	输入
AiChn	音频输入通道号	输入
pstVqeConfig	音频输出声音质量增强配置结构体指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so libAPC_LINUX.so
- 注意
 - 获取声音质量增强功能相关属性前必须先设置相对应AO通道的声音质量增强功能相关属性。

- 在API版本2.17及以后，已不支持此接口。
- 举例
请参考[MI_AO_SetVqeAttr](#) [#320-mi_ao_vqeconfig_t]举例部分。

2.22. MI_AO_EnableVqe

- 功能
使能AO的声音质量增强功能。
- 语法

```
MI_S32 MI_AO_EnableVqe(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn);
```

- 形参
表2-22

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so libAPC_LINUX.so`
- 注意
 - 启用声音质量增强功能前必须先启用相对应的AO通道。
 - 多次使能相同AO通道的声音质量增强功能时，返回成功。

- 禁用AO通道后，如果重新启用AO通道，并使用声音质量增强功能，需调用此接口重新启用声音质量增强功能。
- Vqe包含的所有算法均 支持8/8K 16K采样率，仅ANR/AGC/EQ支持48K。
- 在API版本2.17及以后，已不支持此接口。
- 举例

请参考[MI_AO_SetVqeAttr](#) [#320-mi_ao_vqeconfig_t]举例部分。

2.23. MI_AO_DisableVqe

- 功能
禁用AO的声音质量增强功能。
- 语法

```
MI_S32 MI_AO_DisableVqe(MI_AUDIO_DEV AoDevId,MI_AO_CHN AoChn);
```

- 形参

表2-23

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so libAPC_LINUX.so
- 注意

- 不再使用AO声音质量增强功能时，应该调用此接口将其禁用。
- 多次禁用相同AO 通道的声音质量增强功能，返回成功。
- 在API版本2.17及以后，已不支持此接口。
- 举例

请参考[MI_AO_SetVqeAttr](#) [#320-mi_ao_vqeconfig_t]举例部分。

2.24. MI_AO_SetAdecAttr

- 功能

设置AO解码功能相关属性。

- 语法

```
MI_S32 MI_AO_SetAdecAttr (MI_AUDIO_DEV AoDevId,    MI_AO_CHN
AoChn, MI_AO_AdecConfig_t  *pstAdecConfig);
```

- 形参

表2-24

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入
pstAdecConfig	音频解码配置结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so libg711.a libg726.a

- 注意
 - 设置AO的解码功能相关属性前，必须先使能对应的AO通道。
 - 在API版本2.17及以后，已不支持此接口。
- 举例

下面的代码实现解码功能的设置和使能。

```
1. MI_S32 ret;
2. MI_AUDIO_Attr_t stAttr;
3. MI_AUDIO_Attr_t stGetAttr;
4. MI_AUDIO_DEV AoDevId = 0;
5. MI_AO_CHN AoChn = 0;
6. MI_U8 u8Buf[1024];
7. MI_AUDIO_Frame_t stAoSendFrame;
8. MI_AO_AdecConfig_t stAoSetAdecConfig, stAoGetAdecConfig;
9.
10.
11. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
12. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
13. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
14. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
15. stAttr.u32PtNumPerFrm = 1024;
16. stAttr.u32ChnCnt = 1;
17.
18. MI_SYS_Init();
19.
20. /* set ao public attr*/
21. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
22. if(MI_SUCCESS != ret)
23. {
24.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
25.     return ret;
26. }
27.
28. /* get ao public attr */
29. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
30. if(MI_SUCCESS != ret)
31. {
32.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
33.     return ret;
34. }
35.
36. /* enable ao device*/
37. ret = MI_AO_Enable(AoDevId);
38. if(MI_SUCCESS != ret)
39. {
```

```
40.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
41.     return ret;
42. }
43.
44. /* enable ao chn */
45. ret = MI_AO_EnableChn(AoDevId, AoChn);
46. if (MI_SUCCESS != ret)
47. {
48.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
49.     return ret;
50. }
51.
52. memset(&stAoSetAdecConfig, 0x0,
sizeof(MI_AO_AdecConfig_t));
53. stAoSetAdecConfig.eAdecType = E_MI_AUDIO_ADEC_TYPE_G711A;
54. stAoSetAdecConfig.stAdecG711Cfg.eSamplerate =
E_MI_AUDIO_SAMPLE_RATE_8000;
55. stAoSetAdecConfig.stAdecG711Cfg.eSoundmode =
E_MI_AUDIO_SOUND_MODE_MONO;
56.
57. /* set adec attr */
58. ret = MI_AO_SetAdecAttr(AoDevId, AoChn,
&stAoSetAdecConfig);
59. if (MI_SUCCESS != s32Ret)
60. {
61.     printf("set adec attr ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
62.     return ret;
63. }
64.
65. /* get adec attr */
66. ret = MI_AO_GetAdecAttr(AoDevId, AoChn,
&stAoGetAdecConfig);
67. if (MI_SUCCESS != s32Ret)
68. {
69.     printf("get adec attr ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
70.     return ret;
71. }
72.
73. /* enable adec */
74. ret = MI_AO_EnableAdec(AoDevId, AoChn);
75. if (MI_SUCCESS != s32Ret)
76. {
77.     printf("enable adec ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
78.     return ret;
79. }
```

```

80.
81. /* send frame */
82. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
83. stAoSendFrame.u32Len = 1024;
84. stAoSendFrame.apVirAddr[0] = u8Buf;
85. stAoSendFrame.apVirAddr[1] = NULL;
86.
87. do{
88.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
89. }while(ret == MI_AO_ERR_NOBUF);
90.
91. /* disable adec */
92. ret = MI_AO_DisableAdec(AoDevId, AoChn);
93. if (MI_SUCCESS != s32Ret)
94. {
95.     printf("disable adec ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
96.     return ret;
97. }
98.
99. /* disable ao chn */
100. ret = MI_AO_DisableChn(AoDevId, AoChn);
101. if (MI_SUCCESS != ret)
102. {
103.     printf("disable ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
104.     return ret;
105. }
106.
107. /* disable ao device */
108. ret = MI_AO_Disable(AoDevId);
109. if(MI_SUCCESS != ret)
110. {
111.     printf("disable ao dev %d err:0x%x\n", AoDevId,
ret);
112.     return ret;
113. }
114.
115. MI_SYS_Exit();

```

2.25. MI_AO_GetAdecAttr

- 功能

获取AO解码功能相关属性。

- 语法

```
MI_S32 MI_AO_GetAdecAttr (MI_AUDIO_DEV AoDevId, MI_AO_CHN
AoChn, MI_AO_AdecConfig_t *pstAdecConfig);
```

- 形参

表2-25

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入
pstAdecConfig	音频解码配置结构体指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.a/libmi_ao.so libg711.a libg726.a

- 注意

- 在API版本2.17及以后，已不支持此接口。

- 举例

请参考[MI_AO_SetAdecAttr](#) [224-mi_ao_setadecattr]举例部分。

2.26. MI_AO_EnableAdec

- 功能

使能AO解码功能。

- 语法

```
MI_AO_EnableAdec (MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn);
```

- 形参

表2-26

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so libg711.a libg726.a`

- 注意

- 使能AO的解码功能相关前，必须先设置对应AO通道的解码参数。
- 在API版本2.17及以后，已不支持此接口。

- 举例

请参考[MI_AO_SetAdecAttr](#) [#224-mi_ao_setadecattr]举例部分。

2.27. MI_AO_DisableAdec

- 功能

禁用AO解码功能。

- 语法

```
MI_AO_DisableAdec (MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn);
```

- 形参

表2-27

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so libg711.a libg726.a
- 注意
 - 在API版本2.17及以后，已不支持此接口。
- 举例

请参考[MI_AO_SetAdecAttr](#) [#224-mi_ao_setadecattr]举例部分。

2.28. MI_AO_SetChnParam

- 功能

设置音频通道参数。
- 语法

```
MI_S32 MI_AO_SetChnParam(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_AO_ChnParam_t *pstChnParam);
```

- 形参

表2-28

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号 取值范围：[0, MI_AUDIO_MAX_CHN_NUM [#33-mi_audio_max_chn_num])	输入
pstChnParam	音频通道参数结构体指针	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 举例

下面的代码实现通道参数的设置和获取。

```
1. MI_S32 ret;  
2. MI_AUDIO_Attr_t stAttr;  
3. MI_AUDIO_Attr_t stGetAttr;  
4. MI_AUDIO_DEV AoDevId = 0;  
5. MI_AO_CHN AoChn = 0;  
6. MI_U8 u8Buf[1024];  
7. MI_AUDIO_Frame_t stAoSendFrame;  
8. MI_AO_ChnParam_t stChnParam;  
9.  
10.  
11. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;  
12. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;  
13. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;  
14. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;  
15. stAttr.u32PtNumPerFrm = 1024;  
16. stAttr.u32ChnCnt = 1;
```

```
17.
18. MI_SYS_Init();
19.
20. /* set ao public attr*/
21. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
22. if(MI_SUCCESS != ret)
23. {
24.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
25.     return ret;
26. }
27.
28. /* get ao public attr */
29. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
30. if(MI_SUCCESS != ret)
31. {
32.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
33.     return ret;
34. }
35.
36. /* enable ao device*/
37. ret = MI_AO_Enable(AoDevId);
38. if(MI_SUCCESS != ret)
39. {
40.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
41.     return ret;
42. }
43.
44. /* enable ao chn */
45. ret = MI_AO_EnableChn(AoDevId, AoChn);
46. if (MI_SUCCESS != ret)
47. {
48.     printf("enable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
49.     return ret;
50. }
51.
52. memset(&stChnParam, 0x0, sizeof(stChnParam));
53. stChnParam.stChnGain.bEnableGainSet = TRUE;
54. stChnParam.stChnGain.s16Gain = 0;
55.
56. /* set chn param */
57. ret = MI_AO_SetChnParam(AoDevId, AoChn, &stChnParam);
58. if (MI_SUCCESS != ret)
59. {
60.     printf("set chn param ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
61.     return ret;
62. }
63.
```

```

64. memset(&stChnParam, 0x0, sizeof(stChnParam));
65.
66. ret = MI_AO_GetChnParam(AoDevId, AoChn, &stChnParam);
67. if (MI_SUCCESS != ret)
68. {
69.     printf("get chn param ao dev %d chn %d err:0x%x\n",
AoDevId, AoChn, ret);
70.     return ret;
71. }
72.
73. /* send frame */
74. memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
75. stAoSendFrame.u32Len = 1024;
76. stAoSendFrame.apVirAddr[0] = u8Buf;
77. stAoSendFrame.apVirAddr[1] = NULL;
78.
79. do{
80.     ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame,
-1);
81. }while(ret == MI_AO_ERR_NOBUF);
82.
83. /* disable ao chn */
84. ret = MI_AO_DisableChn(AoDevId, AoChn);
85. if (MI_SUCCESS != ret)
86. {
87.     printf("disable ao dev %d chn %d err:0x%x\n", AoDevId,
AoChn, ret);
88.     return ret;
89. }
90.
91. /* disable ao device */
92. ret = MI_AO_Disable(AoDevId);
93. if(MI_SUCCESS != ret)
94. {
95.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
96.     return ret;
97. }
98.
99. MI_SYS_Exit();

```

2.29. MI_AO_GetChnParam

- 功能

获取音频通道参数。

- 语法

```
MI_S32 MI_AO_GetChnParam(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_AO_ChnParam_t *pstChnParam);
```

- 形参

表2-29

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输出通道号 取值范围：[0, MI_AUDIO_MAX_CHN_NUM [#33-mi_audio_max_chn_num])	输入
pstChnParam	音频通道参数结构体指针	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 举例

请参考[MI_AO_SetChnParam](#) [#228-mi_ao_setchnparam]举例部分。

2.30. MI_AO_SetSrcGain

- 功能

设置AO数据的回声参考增益。
- 语法

```
MI_S32 MI_AO_SetSrcGain(MI_AUDIO_DEV AoDevId, MI_S32
s32VolumeDb);
```

- 形参

表2-30

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
s32VolumeDb	音频设备音量大小 (-60 - 30dB)	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so
- 注意
 - AO设备成功启用后再调用此接口。
- 举例

下面的代码实现设置Src gian。

```
1. MI_S32 ret;
2. MI_AUDIO_Attr_t stAttr;
3. MI_AUDIO_Attr_t stGetAttr;
4. MI_AUDIO_DEV AoDevId = 0;
5. MI_AO_CHN AoChn = 0;
6. MI_U8 u8Buf[1024];
7. MI_AUDIO_Frame_t stAoSendFrame;
8.
9. stAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
10. stAttr.eSamplerate = E_MI_AUDIO_SAMPLE_RATE_8000;
11. stAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
12. stAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
13. stAttr.u32PtNumPerFrm = 1024;
```

```
14. stAttr.u32ChnCnt = 1;
15.
16. MI_SYS_Init();
17.
18. /* set ao public attr*/
19. ret = MI_AO_SetPubAttr(AoDevId, &stAttr);
20. if(MI_SUCCESS != ret)
21. {
22.     printf("set ao %d attr err:0x%x\n", AoDevId, ret);
23.     return ret;
24. }
25.
26. /* get ao public attr */
27. ret = MI_AO_GetPubAttr(AoDevId, &stGetAttr);
28. if(MI_SUCCESS != ret)
29. {
30.     printf("get ao %d attr err:0x%x\n", AoDevId, ret);
31.     return ret;
32. }
33.
34. /* enable ao device*/
35. ret = MI_AO_Enable(AoDevId);
36. if(MI_SUCCESS != ret)
37. {
38.     printf("enable ao dev %d err:0x%x\n", AoDevId, ret);
39.     return ret;
40. }
41.
42. ret = MI_AO_SetSrcGain(AoDevId, 0);
43. if(MI_SUCCESS != ret)
44. {
45.     printf("set src gain ao dev %d err:0x%x\n", AoDevId,
ret);
46.     return ret;
47. }
48.
49. /* disable ao device */
50. ret = MI_AO_Disable(AoDevId);
51. if(MI_SUCCESS != ret)
52. {
53.     printf("disable ao dev %d err:0x%x\n", AoDevId, ret);
54.     return ret;
55. }
56.
57. MI_SYS_Exit();
```

2.31. MI_AO_InitDev

- 功能
初始化AO设备。
- 语法

```
MI_S32 MI_AO_InitDev(MI_AO_InitParam_t *pstInitParam);
```

- 形参

表2-31

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.so/libmi_ao.a
- 注意
 - 本接口必须和[MI_AO_DeinitDev](#) [#232-mi_ao_deinitdev]成对使用，不可单独重复调用，否则返回失败。
 - 仅用于STR状态唤醒后重新初始化AO模块。

2.32. MI_AO_DeInitDev

- 功能
反初始化AO设备。
- 语法

```
MI_S32 MI_AO_DeInitDev(void);
```

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。
- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.so/libmi_ao.a`
- 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在app退出前没有调用，内部会自动反初始化设备。
 - 本接口必须和MI_AO_InitDev成对使用，不可单独重复调用，否则返回失败。
 - 本接口参数设置只在SSC33X生效。

2.33. MI_AO_DupChn

- 功能

同步AO通道状态。
- 语法

```
MI_S32 MI_AO_DupChn(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn)
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号。	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。

- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so

- 注意

此接口仅适用于dual os环境。在rtos环境下已初始化AO通道，切换到Linux环境时，用于同步AO通道的状态。

2.34. MI_AO_SetLRVolume

- 功能

设置AO设备左右声道音量大小。

- 语法

```
MI_AO API Version >= 2.17:  
MI_S32 MI_AO_SetLRVolume(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,  
MI_S32 s32LeftVolumeDb, MI_S32 s32RightVolumeDb);
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号（保留无效）	输入
s32LeftVolumeDb	音频设备左声道音量大小（-60 – 30，以dB为单位）。	输入
s32RightVolumeDb	音频设备右声道音量大小（-60 – 30，以dB为单位）。	输入
eFading	音频增益变化的速度	输入

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [jump]。

- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`

- 注意

AO设备成功启用后再调用此接口。

- 举例

请参考[MI_AO_SetVolume](#) [#215-mi_ao_setvolume]的举例部分。

2.35. MI_AO_GetLRVolume

- 功能

获取AO设备左右声道音量大小。
- 语法

```
MI_AO API Version >= 2.17:  
MI_S32 MI_AO_GetLRVolume(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn,
```

```
MI_S32 *ps32LeftVolumeDb, MI_S32 *ps32RightVolumeDb);
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频通道号（保留无效）	输入
ps32LeftVolumeDb	音频设备左声道音量大小指针	输出
ps32RightVolumeDb	音频设备右声道音量大小指针	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

- 注意

AO设备成功启用后再调用此接口。

- 举例

请参考[MI_AO_SetVolume](#) [#215-mi_ao_setvolume]的举例部分。

2.36. MI_AO_DevIsEnable

- 功能

获取AO设备的使能状态。

- 语法

```
MI_S32 MI_AO_DevIsEnable(MI_AUDIO_DEV AoDevId, MI_BOOL *pbEnable)
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
pbEnable	音频设备使能状态。	输出

- 返回值

- 0 成功。
- 非0 失败，参照[错误码](#) [#jump]。

- 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.a/libmi_ao.so

2.37. MI_AO_ChnIsEnable

- 功能

获取AO通道的使能状态。

- 语法

```
MI_S32 MI_AO_ChnIsEnable(MI_AUDIO_DEV AoDevId, MI_AO_CHN AoChn, MI_BOOL *pbEnable)
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频设备号	输入
AoChn	音频输入通道号。 支持的通道范围由AO设备属性中的最大通道个数 u32ChnCnt决定。	输入
pbEnable	音频通道使能状态通道号。	输出

- 返回值
 - 0 成功。
 - 非0 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_ao.h
 - 库文件: libmi_ao.a/libmi_ao.so

3. AO 数据类型

3.1. AO模块相关数据类型定义

表3-1

数据类型	定义
MI_AUDIO_DEV [#32-mi_audio_dev]	定义音频输入/输出设备编号
MI_AUDIO_MAX_CHN_NUM [#33-mi_audio_max_chn_num]	定义音频输入/输出设备的最大通道数
MI_AO_CHN [#34-mi_ao_chn]	定义音频输出通道
数据类型 MI_AUDIO_SAMPLERATE_E [#35-mi_audio_samplerate_e]	定义音频采样率

mi_audio_samplerate_e]		
MI_AUDIO_Bitwidth_e [#36-mi_audio_bitwidth_e]	定义音频采样精度	
MI_AUDIO_Mode_e [#37-mi_audio_mode_e]	定义音频输入输出工作模式	
MI_AUDIO_SoundMode_e [#38-mi_audio_soundmode_e]	定义音频声道模式	
MI_AUDIO_HpfFreq_e [#39-mi_audio_hpffreq_e]	定义音频高通滤波截止频率	
MI_AUDIO_AdecType_e [#310-mi_audio_adectype_e]	定义音频解码类型	
MI_AUDIO_G726Mode_e [#311-mi_audio_g726mode_e]	定义G726工作模式	
MI_AUDIO_I2sFmt_e [#312-mi_audio_i2sfmt_e]	I2S 格式设定	
MI_AUDIO_I2sMclk_e [#313-mi_audio_i2smclk_e]	I2S MCLK 设定	
MI_AUDIO_I2sConfig_t [#314-mi_audio_i2sconfig_t]	定义I2S属性结构体	
MI_AUDIO_Attr_t [#315-mi_audio_attr_t]	定义音频输入输出设备属性结构体	
MI_AO_ChnState_t [#316-mi_ao_chnstate_t]	音频输出通道的数据缓存状态结构体	
MI_AUDIO_Frame_t [#317-mi_audio_frame_t]	定义音频帧数据结构体	
MI_AUDIO_AecFrame_t [#318-mi_audio_aecframe_t]	定义回声抵消参考帧信息结构体	
MI_AUDIO_SaveFileInfo_t [#319-mi_audio_savefileinfo_t]	定义音频保存文件功能配置信息结构体	
MI_AO_VqeConfig_t [#320-mi_ao_vqeconfig_t]	定义音频输出声音质量增强配置信息结构体	
MI_AUDIO_HpfConfig_t [#321-mi_audio_hpffconfig_t]	定义音频高通滤波功能配置信息结构体	
MI_AUDIO_AnrcConfig_t [#322-mi_audio_anrconfig_t]	定义音频语音降噪功能配置信息结构体	
数据类型		

MI_AUDIO_NrSpeed_e [#323-mi_audio_nrspeed_e]	定义噪声收敛速度
MI_AUDIO_AgcConfig_t [#324-mi_audio_agcconfig_t]	定义音频自动增益控制配置信息结构体
AgcGainInfo_t [#325-agcgaininfo_t]	AGC增益的取值
MI_AUDIO_EqConfig_t [#326-mi_audio_eqconfig_t]	定义音频均衡器功能配置信息结构体
MI_AO_AdecConfig_t [#327-mi_ao_adeconfig_t]	定义解码功能配置信息结构体
MI_AUDIO_AdecG711Config_t [#328-mi_audio_adechg711config_t]	定义G711解码功能配置信息结构体
MI_AUDIO_AdecG726Config_s [#329-mi_audio_adechg726config_s]	定义G711解码功能配置信息结构体
MI_AUDIO_AlgorithmMode_e [#330-mi_audio_algorithmmode_e]	定义音频算法的运行模式
MI_AO_ChnParam_t [#331-mi_ao_chnparam_t]	定义音频通道属性结构体
MI_AO_ChnGainConfig_t [#332-mi_ao_chngainconfig_t]	定义音频通道增益设置结构体
MI_AO_InitParam_t [#333-mi_ao_initparam_t]	定义音频设备初始化参数
MI_AO_GainFading_e [#334-mi_ao_gainfading_e]	定义音频增益的变化速度

3.2. MI_AUDIO_DEV

- 说明

定义音频输入/输出设备编号。

- 定义

```
typedef MI_S32 MI_AUDIO_DEV
```

3.3. MI_AUDIO_MAX_CHN_NUM

- 说明
定义音频输入/输出设备的最大通道数。
- 定义

```
#define MI_AUDIO_MAX_CHN_NUM 16
```

3.4. MI_AO_CHN

- 说明
定义音频输出通道。
- 定义

```
typedef MI_S32 MI_AO_CHN
```

3.5. MI_AUDIO_SampleRate_e

- 说明
定义音频采样率。
- 定义

```
typedef enum  
{  
    E_MI_AUDIO_SAMPLE_RATE_8000 = 8000, /* 8kHz sampling rate  
    */  
    E_MI_AUDIO_SAMPLE_RATE_11052 = 11025, /* 11.025kHz sampling  
    rate */  
}
```

```
E_MI_AUDIO_SAMPLE_RATE_12000 = 12000, /* 12kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_16000 = 16000, /* 16kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_22050 = 22050, /* 22.05kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_24000 = 24000, /* 24kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_32000 = 32000, /* 32kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_44100 = 44100, /* 44.1kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_48000 = 48000, /* 48kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_96000 = 96000, /* 96kHz sampling
rate */

E_MI_AUDIO_SAMPLE_RATE_INVALID,

}MI_AUDIO_SampleRate_e;;
```

- 成员

表3-4

成员名称	描述
E_MI_AUDIO_SAMPLE_RATE_8000	8kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_11025	11.025kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_12000	12kHz采样率
E_MI_AUDIO_SAMPLE_RATE_16000	16kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_22050	22.05kHz采样率
E_MI_AUDIO_SAMPLE_RATE_24000	24kHz采样率
E_MI_AUDIO_SAMPLE_RATE_32000	32kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_44100	44.1kHz采样率
E_MI_AUDIO_SAMPLE_RATE_48000	48kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_96000	96kHz采样率

- 注意事项

这里枚举值不是从0开始，而是与实际的采样率值相同。

- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#315-mi_audio_attr_t]。

3.6. MI_AUDIO_Bitwidth_e

- 说明

定义音频采样精度。

- 定义

```
typedef enum
```

```
{
```

```

E_MI_AUDIO_BIT_WIDTH_16 =0, /* 16bit width */

E_MI_AUDIO_BIT_WIDTH_24 =1, /* 24bit width */

E_MI_AUDIO_BIT_WIDTH_32 =2, /* 32bit width */

E_MI_AUDIO_BIT_WIDTH_MAX,

}MI_AUDIO_BitWidth_e;

```

- 成员

表3-5

成员名称	描述
E_MI_AUDIO_BIT_WIDTH_16	采样精度为16bit 位宽
E_MI_AUDIO_BIT_WIDTH_24	采样精度为24bit 位宽
E_MI_AUDIO_BIT_WIDTH_32	采样精度为32bit 位宽

- 注意事项

目前芯片只支持16bit 位宽，但部分芯片的I2S可以支持32bit收发。

3.7. MI_AUDIO_Mode_e

- 说明

定义音频输入输出设备工作模式。

- 定义

```

typedef enum

{

    E_MI_AUDIO_MODE_I2S_MASTER, /* I2S master mode */

    E_MI_AUDIO_MODE_I2S_SLAVE, /* I2S slave mode */

```

```
E_MI_AUDIO_MODE_TDM_MASTER, /* TDM master mode */

E_MI_AUDIO_MODE_TDM_SLAVE, /* TDM slave mode */

E_MI_AUDIO_MODE_MAX,

}MI_AUDIO_Mode_e;
```

- 成员

表3-6

成员名称	描述
E_MI_AUDIO_MODE_I2S_MASTER	I2S主模式
E_MI_AUDIO_MODE_I2S_SLAVE	I2S从模式
E_MI_AUDIO_MODE_TDM_MASTER	TDM主模式
E_MI_AUDIO_MODE_TDM_SLAVE	TDM从模式

- 注意事项
主模式与从模式是否支持会依据不同的芯片而有区别。
- 相关数据类型及接口
[MI_AUDIO_Attr_t](#) [#315-mi_audio_attr_t]。

3.8. MI_AUDIO_SoundMode_e

- 说明
定义音频声道模式。
- 定义

```
typedef enum

{
```

```
E_MI_AUDIO_SOUND_MODE_MONO =0, /* mono */

E_MI_AUDIO_SOUND_MODE_STEREO =1, /* stereo */

E_MI_AUDIO_SOUND_MODE_QUEUE =2, /* queue */

E_MI_AUDIO_SOUND_MODE_MAX,

}MI_AUDIO_SoundMode_e
```

- 成员

表3-7

成员名称	描述
E_MI_AUDIO_SOUND_MODE_MONO	单声道
E_MI_AUDIO_SOUND_MODE_STEREO	双声道
E_MI_AUDIO_SOUND_MODE_QUEUE	此模式仅应用在音频采集

- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#315-mi_audio_attr_t]

3.9. MI_AUDIO_HpfFreq_e

- 说明

定义音频高通滤波截止频率。

- 定义

```
typedef enum

{

    E_MI_AUDIO_HPF_FREQ_80 = 80, /* 80Hz */

    E_MI_AUDIO_HPF_FREQ_120 = 120, /* 120Hz */

}
```

```
E_MI_AUDIO_HPF_FREQ_150 = 150, /* 150Hz */

E_MI_AUDIO_HPF_FREQ_BUTT,

} MI_AUDIO_HpfFreq_e;
```

- 成员

表3-8

成员名称	描述
E_MI_AUDIO_HPF_FREQ_80	截止频率为80Hz
E_MI_AUDIO_HPF_FREQ_120	截止频率为120Hz
E_MI_AUDIO_HPF_FREQ_150	截止频率为150Hz

- 注意事项

默认配置为150Hz

- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.10. MI_AUDIO_AdecType_e

- 说明

定义音频解码类型。

- 定义

```
typedef enum

{

    E_MI_AUDIO_ADEC_TYPE_G711A = 0,

    E_MI_AUDIO_ADEC_TYPE_G711U,
```

```
E_MI_AUDIO_ADEC_TYPE_G726,

E_MI_AUDIO_ADEC_TYPE_INVALID,

}MI_AUDIO_AdecType_e;
```

- 成员

表3-9

成员名称	描述
E_MI_AUDIO_ADEC_TYPE_G711A	G711A 解码。
E_MI_AUDIO_ADEC_TYPE_G711U	G711U 解码。
E_MI_AUDIO_ADEC_TYPE_G726	G726 解码。

- 相关数据类型及接口

[MI_AO_AdecConfig_t](#) [#327-mi_ao_adeconfig_t]

3.11. MI_AUDIO_G726Mode_e

- 说明

定义G726工作模式。

- 定义

```
typedef enum{

    E_MI_AUDIO_G726_MODE_16 = 0,

    E_MI_AUDIO_G726_MODE_24,

    E_MI_AUDIO_G726_MODE_32,

    E_MI_AUDIO_G726_MODE_40,

    E_MI_AUDIO_G726_MODE_INVALID,
```

```
}MI_AUDIO_G726Mode_e;
```

- 成员

表3-10

成员名称	描述
E_MI_AUDIO_G726_MODE_16	G726 16K 比特率模式。
E_MI_AUDIO_G726_MODE_24	G726 24K 比特率模式。
E_MI_AUDIO_G726_MODE_32	G726 32K 比特率模式。
E_MI_AUDIO_G726_MODE_40	G726 40K 比特率模式

- 相关数据类型及接口

[MI_AO_AdecConfig_t](#) [#327-mi_ao_adeconfig_t]

3.12. MI_AUDIO_I2sFmt_e

- 说明

I2S 格式设定。

- 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_FMT_I2S_MSB,
    E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB,
}MI_AUDIO_I2sFmt_e;
```

- 成员

表3-11

成员名称	描述
E_MI_AUDIO_I2S_FMT_I2S_MSB	I2S 标准格式，最高位优先
E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB	I2S 左对齐格式，最高位优先

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#) [#314-mi_audio_i2sconfig_t]

3.13. MI_AUDIO_I2sMclk_e

- 说明

I2S MCLK 设定

- 定义

```
typedef enum{  
  
    E_MI_AUDIO_I2S_MCLK_0,  
  
    E_MI_AUDIO_I2S_MCLK_12_288M,  
  
    E_MI_AUDIO_I2S_MCLK_16_384M,  
  
    E_MI_AUDIO_I2S_MCLK_18_432M,  
  
    E_MI_AUDIO_I2S_MCLK_24_576M,  
  
    E_MI_AUDIO_I2S_MCLK_24M,  
  
    E_MI_AUDIO_I2S_MCLK_48M,  
  
}MI_AUDIO_I2sMclk_e;
```

- 成员

表3-12

成员名称	描述
E_MI_AUDIO_I2S_MCLK_0	关闭MCLK
E_MI_AUDIO_I2S_MCLK_12_288M	设置MCLK为12.88M
E_MI_AUDIO_I2S_MCLK_16_384M	设置MCLK为16.384M
E_MI_AUDIO_I2S_MCLK_18_432M	设置MCLK为18.432M
E_MI_AUDIO_I2S_MCLK_24_576M	设置MCLK为24.576M
E_MI_AUDIO_I2S_MCLK_24M	设置MCLK为24M
E_MI_AUDIO_I2S_MCLK_48M	设置MCLK为48M

- 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#) [#314-mi_audio_i2sconfig_t]

3.14. MI_AUDIO_I2sConfig_t

- 说明

定义I2S属性结构体。

- 定义

```
typedef struct MI_AUDIO_I2sConfig_s
{
    MI_AUDIO_I2sFmt_e eFmt;

    MI_AUDIO_I2sMclk_e eMclk;

    MI_BOOL bSyncClock;
}MI_AUDIO_I2sConfig_t;
```

- 成员

表3-13

成员名称	描述
eFmt	I2S 格式设置。静态属性。
eMclk	I2S MCLK 时钟频率。静态属性。
bSyncClock	AO 同步AI时钟 静态属性。

- 相关数据类型及接口

[MI_AUDIO_Attr_t](#) [#315-mi_audio_attr_t]

3.15. MI_AUDIO_Attr_t

- 说明
定义音频输入输出设备属性结构体。
- 定义

```
typedef struct MI_AUDIO_Attr_s
{
    MI_AUDIO_SampleRate_e eSamplerate; /*sample rate*/
    MI_AUDIO_BitWidth_e eBitwidth; /*bitwidth*/
    MI_AUDIO_Mode_e eWorkmode; /*master or slave mode*/
    MI_AUDIO_SoundMode_e eSoundmode; /*momo or stereo*/
    MI_U32 u32FrmNum; /*frame num in buffer*/
    MI_U32 u32PtNumPerFrm; /*number of samples*/
    MI_U32 u32CodecChnCnt; /*channel number on Codec */
    MI_U32 u32ChnCnt;
    union{
```

```
MI_AUDIO_I2sConfig_t stI2sConfig;

    }WorkModeSetting;

}MI_AUDIO_Attr_t;
```

- 成员

表3-14

成员名称	描述
eSamplerate	音频采样率。静态属性。
eBitwidth	音频采样精度（从模式下，此参数必须和音频AD/DA 的采样精度匹配）。静态属性。
eWorkmode	音频输入输出工作模式。静态属性。
eSoundmode	音频声道模式。静态属性。
u32FrmNum	缓存帧数目。保留，未使用。
u32PtNumPerFrm	每帧的采样点个数。取值范围为：128, 128*2,...,128*N。静态属性。
u32CodecChnCnt	支持的codec通道数目。保留，未使用。
u32ChnCnt	支持的通道数目，实际可使能的最大通道数。取值：1、2、4、8、16。（输入最多支持MI_AUDIO_MAX_CHN_NUM 个通道，输出最多支持2 个通道）
MI_AUDIO_I2sConfig_t stI2sConfig;	设置I2S 工作属性

- 相关数据类型及接口

[MI_AO_SetPubAttr](#) [#22-mi_ao_setpubattr]

3.16. MI_AO_ChnState_t

- 说明
音频输出通道的数据缓存状态结构体。

- 定义

```
typedef struct MI_AO_ChnState_s
{
```

```
    MI_U32  u32ChnTotalNum;

    MI_U32  u32ChnFreeNum;

    MI_U32  u32ChnBusyNum;
```

```
} MI_AO_ChnState_t;
```

- 成员

表3-15

成员名称	描述
u32ChnTotalNum	输出通道总的缓存字节数。
u32ChnFreeNum	可用的空闲缓存字节数。
u32ChnBusyNum	被占用缓存字节数。

3.17. MI_AUDIO_Frame_t

- 说明
定义音频帧结构体。

- 定义

```
MI_AO API Version <= 2.9:
typedef struct MI_AUDIO_Frame_s
```

```

{
    MI_AUDIO_BitWidth_e eBitwidth;
    MI_AUDIO_SoundMode_e eSoundmode;
    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];
    MI_U64 u64TimeStamp;
    MI_U32 u32Seq;
    MI_U32 u32Len;
    MI_U32 au32PoolId[2];
}MI_AUDIO_Frame_t;

MI_A0 API Version 为 2.9 ~ 2.14:
typedef struct MI_AUDIO_Frame_s
{
    MI_AUDIO_BitWidth_e eBitwidth;
    MI_AUDIO_SoundMode_e eSoundmode;
    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];
    MI_U64 u64TimeStamp;
    MI_U32 u32Seq;
    MI_U32 u32Len;
    MI_U32 au32PoolId[2];
    void *apSrcPcmVirAddr[MI_AUDIO_MAX_CHN_NUM];
    MI_U32 u32SrcPcmLen;
}MI_AUDIO_Frame_t;

MI_A0 API Version >= 2.15:
typedef struct MI_AUDIO_Frame_s
{
    MI_AUDIO_BitWidth_e eBitwidth;
    MI_AUDIO_SoundMode_e eSoundmode;
    void *apVirAddr[MI_AUDIO_MAX_CHN_NUM];
    MI_U64 u64TimeStamp;
    MI_U32 u32Seq;
    MI_U32 u32Len[MI_AUDIO_MAX_CHN_NUM];
    MI_U32 au32PoolId[2];
    void *apSrcPcmVirAddr[MI_AUDIO_MAX_CHN_NUM];
    MI_U32 u32SrcPcmLen[MI_AUDIO_MAX_CHN_NUM];
}MI_AUDIO_Frame_t;

```

- 成员

表3-16

成员名称	描述
eBitwidth	音频采样精度
eSoundmode	音频声道模式
apVirAddr[2]	音频帧数据虚拟地址
u64TimeStamp	音频帧时间戳 以 μs 为单位
u32Seq	音频帧序号
u32Len	音频帧长度 以byte 为单位
u32PoolId[2]	音频帧缓存池ID（保留）
apSrcPcmVirAddr	音频帧原始数据虚拟地址（仅对MI_AI有效）
u32SrcPcmLen	音频帧原始数据的长度（仅对MI_AI有效） 以byte为单位

- 注意事项
 - 当MI_AO API Version ≤ 2.14 ，u32Len（音频帧长度）指整个缓冲区的数据长度。当MI_AO API Version ≥ 2.15 ，每个channel的缓冲数据量为u32Len[channel index]。
 - 单声道数据直接存放，立体声数据按左右声道交错存放，每个声道的采样点数为u32PtNumPerFrm，总的长度为u32Len。

3.18. MI_AUDIO_AecFrame_t

- 说明

定义音频回声抵消参考帧信息结构体。
- 定义

```
typedef struct MI_AUDIO_AecFrame_s
{
    MI_AUDIO_Frame_t stRefFrame; /* aec reference audio frame
    */

    MI_BOOL bValid; /* whether frame is valid */
}MI_AUDIO_AecFrame_t;
```

- 成员

表3-17

成员名称	描述
stRefFrame	回声抵消参考帧结构体。
bValid	参考帧有效的标志。 取值范围： TRUE：参考帧有效。 FALSE：参考帧无效，无效时不能使用此参考帧进行回声抵消。

3.19. MI_AUDIO_SaveFileInfo_t

- 说明

定义音频保存文件功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_SaveFileInfo_s
{
    MI_BOOL bCfg;

    MI_U8 szFilePath[256];

    MI_U32 u32FileSize; /*in KB*/
```

```
} MI_AUDIO_SaveFileInfo_t
```

- 成员

表3-18

成员名称	描述
bCfg	配置使能开关。
szFilePath	音频文件的保存路径
u32FileSize	文件大小，取值范围[1,10240]KB。

- 相关数据类型及接口

[MI_AI_SaveFile](#) [mi_ai.html#MI_AI_SaveFile]

3.20. MI_AO_VqeConfig_t

- 说明

定义音频输出声音质量增强配置信息结构体。

- 定义

```
typedef struct MI_AO_VqeConfig_s
{
    MI_BOOL bHpfOpen;
    MI_BOOL bAnrOpen;
    MI_BOOL bAgcOpen;
    MI_BOOL bEqOpen;
    MI_S32 s32WorkSampleRate;
    MI_S32 s32FrameSample;
```

```

MI_AUDIO_HpfConfig_t stHpfCfg;

MI_AUDIO_AnrcConfig_t stAnrcCfg;

MI_AUDIO_AgcConfig_t stAgcCfg;

MI_AUDIO_EqConfig_t stEqCfg;

} MI_A0_VqeConfig_t;

```

- 成员

表3-19

成员名称	描述
bHpfOpen	高通滤波功能是否使能标志
bAnrcOpen	语音降噪功能是否使能标志
bAgcOpen	自动增益控制功能是否使能标志
bEqOpen	均衡器功能是否使能标志
s32WorkSampleRate	工作采样频率。该参数为内部功能算法工作采样率。取值范围：8kHz/16kHz。默认值为8kHz。
s32FrameSample	VQE的帧长，即采样点数目。只支持128
stHpfCfg	高通滤波功能相关配置信息
stAnrcCfg	语音降噪功能相关配置信息
stAgcCfg	自动增益控制相关配置信息
stEqCfg	均衡器相关配置信息

3.21. MI_AUDIO_HpfConfig_t

- 说明

定义音频高通滤波功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_HpfConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_AUDIO_HpfFreq_e eHpfFreq; /*freq to be processed*/
} MI_AUDIO_HpfConfig_t;
```

- 成员

表3-20

成员名称	描述
eMode	音频算法的运行模式
eHpfFreq	高通滤波截止频率选择 80：截止频率为80Hz； 120：截止频率为120Hz； 150：截止频率为150Hz。 默认值 150

- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.22. MI_AUDIO_AnrcConfig_t

- 说明

定义音频语音降噪功能配置信息结构体。

- 定义

因APC算法库更新，为兼容新旧版本的算法库，以下为不同版本对应的数据结构（可通过strings libAPC_LINUX.so | grep APCSET来获取APC算法库的版本，其中APC API版本会记录在APIVERSION字段中）

APC API VERSION:1（或没有APIVERSION字段）

```
typedef struct MI_AUDIO_AnrcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_U32 u32NrIntensity;

    MI_U32 u32NrSmoothLevel;

    MI_AUDIO_NrSpeed_e eNrSpeed;
} MI_AUDIO_AnrcConfig_t;
```

APC API VERSION:2

```
typedef struct MI_AUDIO_AnrcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    MI_U32 u32NrIntensityBand[6];

    MI_U32 u32NrIntensity[7];

    MI_U32 u32NrSmoothLevel;

    MI_AUDIO_NrSpeed_e eNrSpeed;
} MI_AUDIO_AnrcConfig_t;
```

- 成员

表3-21

APC API VERSION:1（或没有APIVERSION字段）

成员名称	描述
eMode	音频算法的运行模式 注：Anr的模式选择将会在一定程度上影响Agc的功能
u32NrIntensity	降噪力度配置，配置值越大降噪力度越高，但同时也会带来细节的丢失/损伤。 范围[0,30]；步长1 默认值 20。
u32NrSmoothLevel	平滑化程度，值越大越平滑 范围[0,10]；步长1 默认值10。
eNrSpeed	噪声收敛速度，低速，中速，高速 默认值中速。

APC API VERSION:2

成员名称	描述
eMode	音频算法运行模式 注：Anr的模式选择将会在一定程度上影响Agc的功能
u32NrIntensityBand	降噪频率范围，后1个数据必须大于等于前1个数据。 如：u32NrIntensityBand[0] = 10, 则：u32NrIntensityBand[1]必须大于等于10。 当前采样率对应的最高频率平均分成128份，频率范围则是对应多少份组成一个频带。 如：当前采样率为16K，对应的最大频率为8K，每一份为 $8000 / 128 \approx 62.5\text{Hz}$ 。如在{4,6,36, 49,50,51}的设定下，降噪频率范围为{ $0 \sim 4 * 62.5\text{Hz}$, $4 \sim 6 * 62.5\text{Hz}$, $6 \sim 36 * 62.5\text{Hz}$, $36 \sim 49 * 62.5\text{Hz}$, $49 \sim 50 * 62.5\text{Hz}$, $50 \sim 51 * 62.5\text{Hz}$, $51 \sim 127 * 62.5\text{Hz}$ } = {0~250Hz, 250~375Hz, 375~2250Hz, 2250~3062.5Hz, 3062.5~3125Hz, 3125~3187.5Hz, 3187.5Hz~8000Hz} 范围[1,127]；步长1
u32NrIntensity	降噪力度配置，配置值越大降噪力度越高，但同时也会带来细节音的丢失/损伤。范围[0,30]； 步长1 默认值 20
u32NrSmoothLevel	平滑化程度 范围[0,10]； 步长1 默认值10
eNrSpeed	噪声收敛速度，低速，中速，高速 默认值中速

- 注意事项
 - 在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。
- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.23. MI_AUDIO_NrSpeed_e

- 说明
定义噪声收敛速度
- 定义

```
typedef enum
{
    E_MI_AUDIO_NR_SPEED_LOW,
    E_MI_AUDIO_NR_SPEED_MID,
    E_MI_AUDIO_NR_SPEED_HIGH
}MI_AUDIO_NrSpeed_e;
```

- 成员
表3-22

成员名称	描述
E_MI_AUDIO_NR_SPEED_LOW	低速。
E_MI_AUDIO_NR_SPEED_MID	中速。
E_MI_AUDIO_NR_SPEED_HIGH	高速。

- 相关数据类型及接口
[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.24. MI_AUDIO_AgcConfig_t

- 说明
定义音频自动增益控制配置信息结构体。
- 定义

因APC算法库更新，为兼容新旧版本的算法库，以下为不同版本对应的数据结构（可通过strings libAPC_LINUX.so | grep APCSET来获取APC算法库的版本，其中APC API版本会记录在APIVERSION字段中）

APC API VERSION:1（或没有APIVERSION字段）

```
typedef struct MI_AUDIO_AgcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    AgcGainInfo_t stAgcGainInfo;

    MI_U32 u32DropGainMax;

    MI_U32 u32AttackTime;

    MI_U32 u32ReleaseTime;

    MI_S16 s16Compression_ratio_input[5];

    MI_S16 s16Compression_ratio_output[5];

    MI_S32 s32TargetLevelDb;

    MI_S32 s32NoiseGateDb;

    MI_U32 u32NoiseGateAttenuationDb;

} MI_AUDIO_AgcConfig_t;
```

APC API VERSION:2

```
typedef struct MI_AUDIO_AgcConfig_s
{
    MI_AUDIO_AlgorithmMode_e eMode;

    AgcGainInfo_t stAgcGainInfo;

    MI_U32 u32DropGainMax;

    MI_U32 u32AttackTime;
```

```
MI_U32      u32ReleaseTime;

MI_S16      s16Compression_ratio_input[7];

MI_S16      s16Compression_ratio_output[7];

MI_S32      s32DropGainThreshold;

MI_S32      s32NoiseGateDb;

MI_U32      u32NoiseGateAttenuationDb;

} MI_AUDIO_AgcConfig_t;
```

- 成员

表3-23

APC API VERSION:1 (或没有APIVERSION字段)

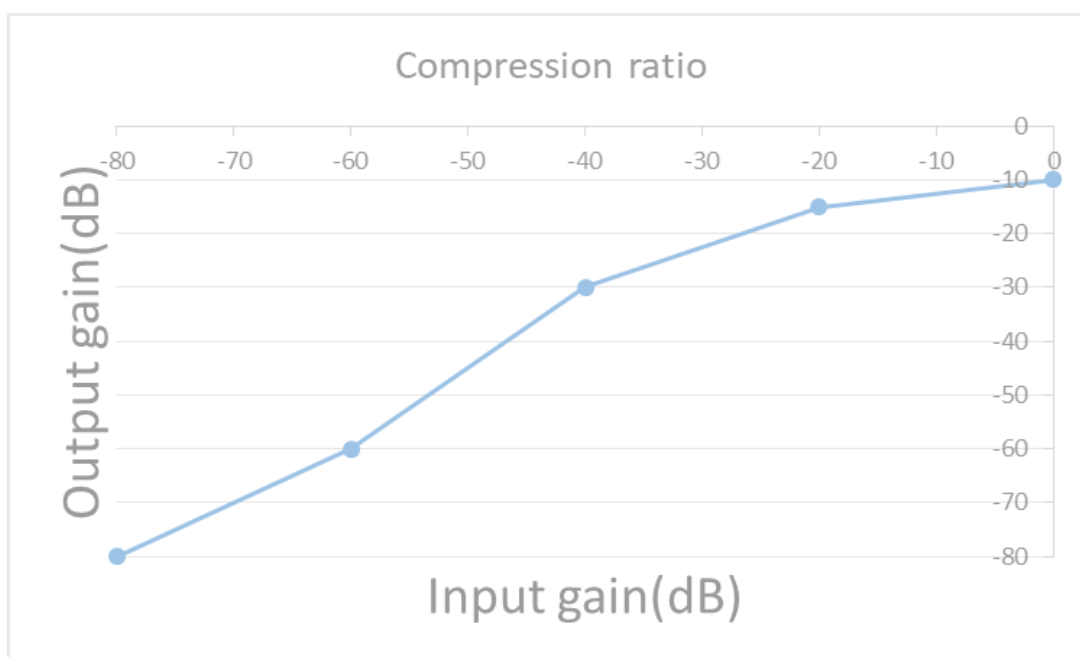
成员名称	描述
eMode	音频算法的运行模式
stAgcGainInfo	定义AGC增益的最大、最小和初始值
u32DropGainMax	增益下降的最大值，防止输出饱和。 范围[0,60]；步长1 默认值55。 注意：此值仅代表能下降的范围，但具体能下降到何值还需参考AGC增益的最小值。
u32AttackTime	增益下降的时间步长，以16毫秒为1单位 范围[1,20]； 步长1 默认值0。
u32ReleaseTime	增益增加的时间步长，以16毫秒为1单位 范围[1, 20]； 步长1 默认值 0。
s16Compression_ratio_input	输入压缩比，必须配合s16Compression_ratio_output使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s16Compression_ratio_output	输出压缩比，必须配合s16Compression_ratio_input使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s32TargetLevelDb	目标电平，经过处理后最大能达到的门限电平。 范围[-80,0]dB；步长1 默认值0。 注意：此值仅能决定最大能达到的电平，但是否能达到此值，还需参考斜率曲线的设置。
s32NoiseGateDb	噪声阈值，当信号小于此值时，当作噪声处理 范围[-80,0]；步长1 默认值-55。 注意：当值为-80，噪声阈值将不起作用
u32NoiseGateAttenuationDb	当噪声阈值起效果时，输入源的衰减百分比 范围[0,100]；步长1 默认值0。

APC API VERSION:2

成员名称	描述
eMode	音频算法的运行模式
stAgcGainInfo	定义AGC增益的最大、最小和初始值
u32DropGainMax	增益下降的最大值，防止输出饱和。 范围[0,60]；步长1 默认值55。 注意：此值仅代表能下降的范围，但具体能下降到何值还需参考AGC增益的最小值。
u32AttackTime	增益下降时间步长，以16毫秒为1单位 范围[1,20]；步长1 默认值0。
u32ReleaseTime	增益增加时间步长，以16毫秒为1单位 范围[1, 20]；步长1 默认值0。
s16Compression_ratio_input	输入压缩比，必须配合s16Compression_ratio_output使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s16Compression_ratio_output	输出压缩比，必须配合s16Compression_ratio_input使用，透过多个转折点实现多斜率的曲线 范围[-80,0]dBFS；步长1
s32DropGainThreshold	衰减阈值，当信号幅度超过此值后，会慢慢进行衰减。 范围[-80,0]dB；步长1 默认值0。
s32NoiseGateDb	噪声阈值，当信号小于此值时，当作噪声处理 范围[-80,0]；步长1 默认值-55。 注意：当值为-80，噪声阈值将不起作用
u32NoiseGateAttenuationDb	当噪声阈值起效果时，输入源的衰减百分比 范围[0,100]；步长1 默认值0。

- 注意事项

- 在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。而s16Compression_ratio_input和s16Compression_ratio_output则需要根据所需要的增益曲线来设定。
- 如下面的折线图所示，在输入增益为-80~0dB划分为四 (APIVERSION 1 或没有APIVERSION字段) /六 (APIVERSION 2) 段斜率，-80dB~-60dB范围内保持原来的增益，斜率为1，-60dB~-40dB范围内需要稍微提高增益，斜率为1.5，-40dB~-20dB范围内斜率为1.25，-20dB~0dB范围内斜率为0.25。根据曲线的转折点对s16Compression_ratio_input和s16Compression_ratio_output设置，若不需要那么多段曲线，则将数组不需要的部分填0。



- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.25. AgcGainInfo_t

- 说明

AGC增益的取值

- 定义

```
typedef struct AgcGainInfo_s{  
  
    MI_S32 s32GainMax;  
  
    MI_S32 s32GainMin;  
  
    MI_S32 s32GainInit;  
  
}AgcGainInfo_t;
```

- 成员

表3-24

成员名称	描述
s32GainMax	增益最大值 范围[0,60]；步长1 默认值15。
s32GainMin	增益最小值 范围[-20,30]；步长1 默认值0。
s32GainInit	增益初始值 范围[-20,60]；步长1 默认值0。

- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [#320-mi_ao_vqeconfig_t]

3.26. MI_AUDIO_EqConfig_t

- 说明

定义音频均衡器功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_EqConfig_s  
  
{  
  
    MI_AUDIO_AlgorithmMode_e eMode;
```

```
MI_S16 s16EqGainDb[129];

} MI_AUDIO_EqConfig_t;
```

- 成员

表3-25

成员名称	描述
eMode	音频算法的运行模式
s16EqGainDb[129]	均衡器增益调节取值，将当前采样率的频率范围分成129个频率范围来进行调节 范围[-50,20]；步长1 默认值0。 如：当前采样率为16K，对应的最高频率为8K， $8000 / 129 \approx 62\text{Hz}$ ，则单个调节的频率范围为62Hz，将0-8K划分成 $\{0-1 * 62\text{Hz}, 1-2 * 62\text{Hz}, 2-3 * 62\text{Hz}, \dots, 128-129 * 62\text{Hz}\} = \{0-62\text{Hz}, 62-124\text{Hz}, 124-186\text{Hz}, \dots, 7938-8000\text{Hz}\}$ ，每段对应一个增益值

- 相关数据类型及接口

[MI_AO_VqeConfig_t](#) [[#320-mi_ao_vqeconfig_t](#)]

3.27. MI_AO_AdecConfig_t

- 说明

定义解码功能配置信息结构体。

- 定义

```
typedef struct MI_AO_AdecConfig_s
{
    MI_AUDIO_AdecType_e eAdecType;

    union
    {
        MI_AUDIO_AdecG711Config_t stAdecG711Cfg;
```

```
MI_AUDIO_AdecG726Config_s stAdecG726Cfg;

};

}MI_A0_AdecConfig_t;
```

- 成员

表3-26

成员名称	描述
eAdecType	音频解码类型
stAdecG711Cfg	G711解码相关配置信息
stAdecG726Cfg	G726解码相关配置信息

- 相关数据类型及接口

[MI_A0_SetAdecAttr](#) [#224-mi_ao_setadecattr]

3.28. MI_AUDIO_AdecG711Config_t

- 说明

定义G711解码功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_AdecG711Config_s{

    MI_AUDIO_SampleRate_e eSamplerate;

    MI_AUDIO_SoundMode_e eSoundmode;

}MI_AUDIO_AdecG711Config_t;
```

- 成员

表3-27

成员名称	描述
eSamplerate	音频采样率
eSoundmode	音频声道模式

- 相关数据类型及接口

[MI_AO_SetAdecAttr](#) [#224-mi_ao_setadecattr]

3.29. MI_AUDIO_AdecG726Config_s

- 说明

定义G711解码功能配置信息结构体。

- 定义

```
typedef struct MI_AUDIO_AdecG726Config_s{  
  
    MI_AUDIO_SampleRate_e eSamplerate;  
  
    MI_AUDIO_SoundMode_e eSoundmode;  
  
    MI_AUDIO_G726Mode_e eG726Mode;  
  
}MI_AUDIO_AdecG726Config_t;
```

- 成员

表3-28

成员名称	描述
eSamplerate	音频采样率
eSoundmode	音频声道模式
eG726Mode	G726工作模式

- 相关数据类型及接口

[MI_AO_SetAdecAttr](#) [#224-mi_ao_setadecattr]

3.30. MI_AUDIO_AlgorithmMode_e

- 说明

音频算法运行的模式。

- 定义

```
typedef enum
{
    E_MI_AUDIO_ALGORITHM_MODE_DEFAULT,
    E_MI_AUDIO_ALGORITHM_MODE_USER,
    E_MI_AUDIO_ALGORITHM_MODE_MUSIC,
    E_MI_AUDIO_ALGORITHM_MODE_INVALID,
} MI_AUDIO_AlgorithmMode_e;
```

- 成员

表3-29

成员名称	描述
E_MI_AUDIO_ALGORITHM_MODE_DEFAULT	默认运行模式 当使用该模式时，将使用算法的默认参数
E_MI_AUDIO_ALGORITHM_MODE_USER	用户模式 当使用该模式时，需要用户重新设定所有参数
E_MI_AUDIO_ALGORITHM_MODE_MUSIC	音乐模式 仅有Anr具有此模式，当为此模式时，Agc不会进行speech enhancement（语音增强）处理

- 注意事项

在Anr和Agc都有使能的情况下，当Anr设定为user mode时，Agc会对音频数据做频域处理，会评估出语音信号再做相应的增减，而当Anr设定为default/music mode时，Agc会对音频数据做时域处理，对全频段的数据进行增减。

- 相关数据类型及接口

[MI_AUDIO_HpfConfig_t](#) [#321-mi_audio_hpfconfig_t]

[MI_AUDIO_AnrcConfig_t](#) [#322-mi_audio_anrconfig_t]

[MI_AUDIO_AgcConfig_t](#) [#324-mi_audio_agcconfig_t]

[MI_AUDIO_EqConfig_t](#) [#326-mi_audio_eqconfig_t]

3.31. MI_AO_ChnParam_t

- 说明

定义音频通道参数设置结构体。

- 定义

```
typedef struct MI_AO_ChnParam_s
{
```

```
MI_AO_ChnGainConfig_t) stChnGain;

MI_U32 u32Reserved;

} MI_AO_ChnParam_t;
```

- 成员

表3-30

成员名称	描述
stChnGain	音频通道增益设置结构体
u32Reserved	保留，不使用

- 相关数据类型及接口

[MI_AO_SetChnParam](#) [#228-mi_ao_setchnparam]

[MI_AO_GetChnParam](#) [#229-mi_ao_getchnparam]

3.32. MI_AO_ChnGainConfig_t

- 说明

定义音频通道增益设置结构体。

- 定义

```
typedef struct MI_AO_ChnGainConfig_s
{
    MI_BOOL bEnableGainSet;

    MI_S16 s16Gain;
}MI_AO_ChnGainConfig_t;
```

- 成员

表3-31

成员名称	描述
bEnableGainSet	是否使能增益设置
s16Gain	增益 (-60 – 30dB)

- 相关数据类型及接口

[MI_AO_ChnParam_t](#) [#331-mi_ao_chnparam_t]

3.33. MI_AO_InitParam_t

- 说明

AO设备初始化参数。

- 定义

```
typedef struct MI_AO_InitParam_s
{
    MI_U32  u32DevId;

    MI_U8  *u8Data;
} MI_AO_InitParam_t;
```

- 成员

表3-32

成员名称	描述
u32DevId	音频设备号
u8Data	参数指针 (保留)

- 相关数据类型及接口

[MI_AO_InitDev](#) [#231-mi_ao_initdev]

3.34. MI_AO_GainFading_e

- 说明

AO增益的变化速度。

- 定义

```
typedef enum{  
  
    E_MI_AO_GAIN_FADING_OFF = 0,  
  
    E_MI_AO_GAIN_FADING_1_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_2_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_4_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_8_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_16_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_32_SAMPLE,  
  
    E_MI_AO_GAIN_FADING_64_SAMPLE,  
  
}MI_AO_GainFading_e;
```

- 成员

成员名称	描述
E_MI_AO_GAIN_FADING_OFF	关闭Fading功能，设置增益立刻生效
E_MI_AO_GAIN_FADING_1_SAMPLE	开启Fading功能，1个采样点变化0.5dB
E_MI_AO_GAIN_FADING_2_SAMPLE	开启Fading功能，2个采样点变化0.5dB
E_MI_AO_GAIN_FADING_4_SAMPLE	开启Fading功能，4个采样点变化0.5dB
E_MI_AO_GAIN_FADING_8_SAMPLE	开启Fading功能，8个采样点变化0.5dB
E_MI_AO_GAIN_FADING_16_SAMPLE	开启Fading功能，16个采样点变化0.5dB
E_MI_AO_GAIN_FADING_32_SAMPLE	开启Fading功能，32个采样点变化0.5dB
E_MI_AO_GAIN_FADING_64_SAMPLE	开启Fading功能，64个采样点变化0.5dB

- 相关数据类型及接口

[MI_AO_SetVolume](#) [#215-mi_ao_setvolume]

4. 错误码

AO API 错误码如表4-1所示：

表4-1

错误码	宏定义	描述
0xA0052001	MI_AO_ERR_INVALID_DEVID	音频输出设备号无效
0xA0052002	MI_AO_ERR_INVALID_CHNID	音频输出信道号无效
0xA0052017	MI_AO_ERR_NOT_ENABLED	音频输出设备或信道没有使能
0xA0052006	MI_AO_ERR_NULL_PTR	输入参数空指标错误
0xA0052007	MI_AO_ERR_NOT_CONFIG	音频输出设备属性未设置
0xA0052008	MI_AO_ERR_NOT_SUPPORT	操作不支持
0xA0052009	MI_AO_ERR_NOT_PERM	操作不允许
0xA005200C	MI_AO_ERR_NOMEM	分配内存失败
0xA005200D	MI_AO_ERR_NOBUF	音频输出缓存不足
0xA005200E	MI_AO_ERR_BUF_EMPTY	音频输出缓存为空
0xA005200F	MI_AO_ERR_BUF_FULL	音频输出缓存为满
0xA0052010	MI_AO_ERR_SYS_NOTREADY	音频输出系统未初始化
0xA0052012	MI_AO_ERR_BUSY	音频输出系统忙碌
0xA0052120	MI_AO_ERR_VQE_ERR	音频输出VQE算法处理失败
0xA0052121	MI_AO_ERR_ADEC_ERR	音频输出解码算法处理失败

5. PROCFS介绍

5.1. cat

- 调试信息

```
# cat proc/mi_modules/mi_ao/mi_ao0

===== MI A0 Dev0 Info =====
----- Start A0 Dev0 Attr -----
DevStatus   Format   SoundMode  SampleRate  PeriodSize  ChannelMode
opened      S16_LE  Stereo      8000         2048         Stereo

DmaBufSize  TmpBufSize  bStart  bPause  FrmCnt
65536       65536       1        1       299
DpgaGain    DpgaMute
(-20,-20)  (0,0)
MI If:  DAC_AB
Mhal Left If:  DAC_A_0
Mhal Right If:  DAC_B_0
If Info:
DAC_AB:      Volume( 0, 0) Mute(0,0).
DAC_CD:      Volume( 0, 0) Mute(0,0).
I2S_A:       Volume( 0, 0) Mute(0,0).
I2S_B:       Volume( 0, 0) Mute(0,0).
ECHO_A:      Volume( 0, 0) Mute(0,0).
HDMI_A:      Volume( 0, 0) Mute(0,0).
----- End A0 Dev0 Attr -----
```

- 调试信息分析

记录当前AO的使用状况以及device属性等，可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
AO Device Attr	DevStatus	AO设备状态 uninit:未初始化 opened:open成功
	Format	音频格式（位宽及大小端等） 目前仅支持S16_LE（16bit，小端模式）
	SoundMode	声音模式 mono：单声道 stereo：立体声
	SampleRate	采样率 8k/11.025k/12k/16k/22.05k/24k/32k/44.1k/48k

	PeriodSize	AO起播水线（采样点数）
	ChannelMode	AO通道模式 Stereo:立体声 DoubleMono:目前仅支持，播MONO-左右声道输出相同 DoubleLeft:输出两路左声道 DoubleRight：输出两路右声道 Exchange:左右声道交换 OnlyLeft:目前仅支持MONO-左声道输出 OnlyRight：目前仅支持MONO-右声道输出
	DmaBufSize	DMA buffer大小
	TmpBufSize	Tmp临时buffer大小
	bStart	AO是否开启DMA
	bPause	AO是否暂停DMA
	FrmCnt	写数据帧计数
	DpgaGain	数字增益：（左声道增益，右声道增益）
	DpgaMute	DPGA静音：（左声道静音，右声道静音）
	MI If	MI绑定的所有interface信息，即应用层绑定的所有interface信息
	Mhal Left If	MHAL RDMA_L绑定的所有interface信息
	Mhal Right If	MHAL RDMA_R绑定的所有interface信息
	If Info	Interface音量、静音等信息 Volume（左声道增益，右声道增益） Mute（左声道静音，右声道静音）
AO I2S Status	I2sMode	I2S Tx的工作模式(仅interface为I2S Tx时有效) i2s-master i2s-slave tdm-master tdm-slave
	I2sMclk	I2S Tx的Mclk的频率(仅interface为I2S Tx时有效)

	disable: 不使用Mclk 其他值为当前的Mclk频率
I2sFmt	I2S Tx的数据格式(仅interface为I2S Tx时有效) I2S-MSB: I2S格式 LEFT-MSB: I2S左对齐格式
bl2sSync	I2S RX和TX是否共用clock(仅interface为I2S Tx时有效) 1: 4 wire mode · RX和TX共用clock 0: 6 wire mode · RX和TX都有独立的clock
TdmSlots	I2S Tx的TDM slot数目(仅interface为I2S Tx为TDM模式时有效)
I2sBitWidth	I2S TX的位宽(仅interface为I2S Tx · 且需要支持TDM模式的芯片有效)

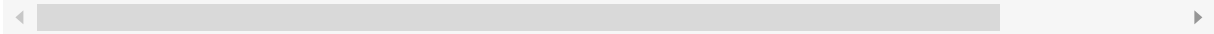
5.2. echo

功能	动态启用/关闭AO设备DPGA静音模式
命令	echo set_dpga_mute [LeftMute] [RightMute] [Fading] > proc/mi_modules/mi_ao/mi_
参数说明	[ON/on/1, OFF/off/0] 开启/关闭静音 [Fading]设置渐入渐出速度 [ID] 设备号
举例	echo set_dpga_mute 1 0 0 > proc/mi_modules/mi_ao/mi_ao[ID]

功能	动态启用/关闭AO设备Interface静音模式
命令	echo set_inf_mute [If] [LeftMute] [RightMute] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[If]需要设定的Interface [ON/on/1, OFF/off/0] 开启/关闭静音 [ID] 设备号
举例	echo set_inf_mute 1 1 0 > proc/mi_modules/mi_ao/mi_ao[ID]



功能	动态修改AO DPGA音量大小
命令	echo set_dpga_volume [LeftVolume] [RightVolume] [Fading] > proc/mi_modules/mi_
参数说明	[LeftVolume] [RightVolume] 左声道音量，右声道音量 [Fading]设置音量渐入渐出速度
举例	echo set_dpga_volume -10 -10 0 > proc/mi_modules/mi_ao/mi_ao0



功能	动态修改AO Interface音量大小
命令	echo set_inf_volume [If] [LeftVolume] [RightVolume] > proc/mi_modules/mi_ao/mi_ao
参数说明	[If]需要设定的Interface [LeftVolume] [RightVolume] 左声道音量，右声道音量
举例	暂不支持



功能	动态开启/关闭AO设备dump数据功能
命令	echo dump_data [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[Path] dump数据的保存路径 [ON/on/1, OFF/off/0] 开启/关闭dump数据功能
举例	echo dump /mnt 1 > proc/mi_modules/mi_ao/mi_ao0

